

Unidad N°1 – Introducción a los Sistemas Operativos

Sistema operativo (SO): es un programa que controla la ejecución de los programas de aplicación y que actúa como interfaz entre las aplicaciones del usuario y el hardware de una computadora.

Funciones y objetivos de los sistemas operativos

Un SO tiene tres objetivos:

- **Comodidad:** hacer que una computadora sea más cómoda y fácil de utilizar.
- **Eficiencia:** permite que los recursos de un sistema informático se aprovechen eficientemente.
- **Capacidad de evolución:** deben construirse de modo que permitan el desarrollo efectivo, la verificación y la introducción de nuevas funciones al sistema, y a la vez, no interferir en los servicios que brinda.

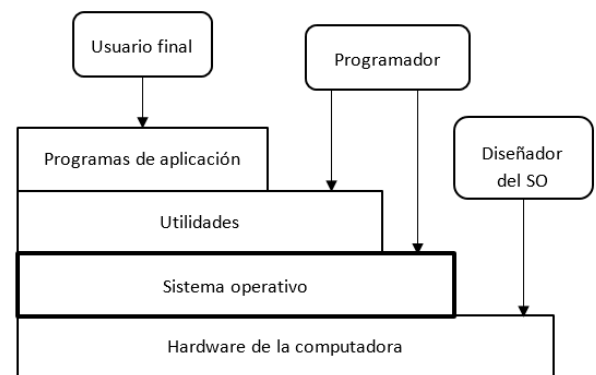
El sistema operativo como interfaz usuario/computadora

Los usuarios finales utilizan programas de aplicación, y no tienen que preocuparse de la arquitectura de la computadora.

Las aplicaciones se construyen con un lenguaje de programación y son desarrolladas por programadores.

Algunos programas se denominan utilidades, que implementan funciones muy utilizadas que ayudan a la creación de programas, la gestión de archivos y el control de dispositivos de E/S.

El SO oculta al programador los detalles de hardware y le proporciona una interfaz cómoda para utilizar el sistema. Actúa como mediador, facilitándole al programador y a los programas de aplicación el acceso y uso de todas esas características y servicios.



Un SO ofrece servicios en las siguientes áreas:

- **Creación de programas:** el SO ofrece múltiples funcionalidades y servicios para ayudar al programador en la creación de programas. Estos servicios están en forma de programas de utilidades.
- **Ejecución de programas:** el SO administra todas las tareas necesarias para la ejecución de un programa.
- **Acceso a los dispositivos de E/S:** el SO brinda una interfaz uniforme que oculta los detalles de funcionamiento de los dispositivos de E/S, de modo que el programador pueda acceder a los dispositivos utilizando lecturas y escrituras simples.
- **Acceso controlado a los archivos:** el SO proporciona mecanismos de protección para controlar el acceso a los archivos.
- **Acceso al sistema:** en un sistema compartido o público, el SO controla el acceso al sistema como un todo y a los recursos específicos del sistema.
- **Detección y respuesta a errores:** el SO debe dar respuestas que elimine las condiciones de errores que se produzcan con el menor impacto posible sobre las aplicaciones que están ejecución.
- **Contabilidad:** el SO debe recoger estadísticas de los recursos y supervisar parámetros de rendimiento. Es muy útil para anticiparse a la necesidad de mejoras futuras y para ajustar el sistema y así mejorar su rendimiento.

El sistema operativo como administrador de recursos

Una computadora es un conjunto de recursos para el movimiento, almacenamiento y proceso de datos y para el control de estas funciones. El SO es el responsable de la gestión de estos recursos.

El SO es nada más que un conjunto de programas, que como otros programas, proporciona instrucciones al procesador.

El SO dirige al procesador en el empleo de otros recursos del sistema y en el control del tiempo de la ejecución de otros programas. Pero para que el procesador pueda hacer estas cosas, debe cesar la ejecución del programa del SO y ejecutar otros programas. El SO cede el control del procesador, para realizar algún trabajo y luego lo recupera durante el tiempo suficiente para preparar al procesador para llevar a cabo la siguiente parte del trabajo.

Una parte del SO está en la memoria principal. En esta parte está el **núcleo** (*kernel*), que incluye las funciones más utilizadas frecuentemente del SO. El resto de la memoria principal contiene datos y otros programas de usuario. La asignación de la memoria principal es controlada conjuntamente por el SO y por el hardware de gestión de memoria del procesador. El SO decide cuándo un programa en ejecución puede utilizar un dispositivo de E/S y controla el acceso y la utilización de los archivos. El procesador es un recurso y es el SO el que debe determinar cuánto tiempo del procesador debe dedicarse a la ejecución de un programa de usuario en particular.

Facilidad de evolución de un sistema operativo

Un SO evoluciona en el tiempo por las siguientes razones:

- Actualizaciones de hardware y nuevos tipos de hardware.
- Nuevos servicios, como respuesta a las demandas de los usuarios o necesidades de administradores de sistemas.
- Correcciones de fallos que se descubren con el tiempo.

La necesidad de hacer cambios en forma regular, introduce ciertos requisitos de diseño. El sistema debe tener una construcción modular, con interfaces bien definidas entre los módulos y debe estar bien documentado.

Evolución histórica de los sistemas operativos

- ❖ Procesos en serie.
- ❖ Sistemas sencillos de proceso por lotes.
- ❖ Sistemas por lotes con multiprogramación.
- ❖ Sistemas de tiempo compartido.

Proceso en serie

En las primeras computadoras los programadores interactuaban directamente con el hardware de los mismos, no había sistema operativo. Se operaba desde una consola consistente en unos indicadores luminosos, unos conmutadores, algún tipo de dispositivo de entrada y alguna impresora.

Los programas en código máquina se cargaban con un dispositivo de entrada (lector de tarjetas). Se ejecutaban hasta su culminación donde la impresora mostraba el resultado, si se producía un error se mostraba con los indicadores luminosos.

Presentaban dos problemas principales:

- **Planificación:** la mayoría de las instalaciones empleaban un formulario de reserva de tiempo de máquina. Si un usuario reservaba una hora y terminaba a los 45 minutos se desperdiciaba tiempo del computador; por el contrario, si el usuario tenía dificultades, no terminaba en el tiempo asignado y tenía que parar sin resolver el problema.
- **Tiempo de preparación:** si se producía un error, el usuario tenía que volver a realizar el proceso de preparación perdiendo tiempo considerable. El proceso de preparación consistía en cargar compilador y programa en alto nivel en memoria, salvar el programa objeto (compilado) y luego cargarlo y montarlo junto con las funciones comunes.

Se llamaba proceso en serie porque los usuarios accedían al computador en serie.

Sistemas sencillos de proceso por lotes

El primer sistema operativo por lotes (y el primer sistema operativo de todos) se desarrolló a mediados de los 50 por General Motors.

La idea central es el uso de un elemento de software conocido como el **monitor**. Aquí, los usuarios no tenían acceso directo a la máquina. En su lugar, el usuario debía entregar los trabajos en tarjetas o en cintas al operador de la computadora, quien agrupaba secuencialmente los trabajos por lotes y ubicaba los lotes enteros en un dispositivo de entrada para su empleo por parte del monitor. Cada programa se construía de modo que al terminar su ejecución, el monitor cargaba automáticamente el siguiente programa.

El monitor es quien gestiona el problema de la planificación. Se pone en cola un lote de trabajos y éstos son ejecutados tan rápidamente como sea posible, evitando tiempos de inactividad. El monitor también organiza mejor el tiempo de organización del trabajo utilizando un lenguaje de control de trabajos, que es un lenguaje para dar instrucciones al monitor.

Este tipo de sistemas necesitaban nuevas características como:

- **Protección de memoria:** mientras se ejecuta un programa de usuario no debe modificar la zona de memoria donde está el monitor.
- **Temporizador:** se utiliza para evitar que un solo trabajo monopolice el sistema.
- **Instrucciones privilegiadas:** algunas instrucciones máquina solo las puede ejecutar el monitor.
- **Interrupciones:** le flexibilidad al sistema operativo para ceder y retomar el control de los programas de usuario.

Sistemas por lotes con multiprogramación

Aún con el secuenciamiento automático de trabajos ofrecido por un sistema operativo sencillo por lotes, el procesador está desocupado con frecuencia, ya que los dispositivos de E/S son lentos comparados al microprocesador.

Consiste en tener varios programas de usuario en la memoria, y cuando el programa que está en ejecución va esperar por E/S, el procesador cambia a otro trabajo que no esté esperando por E/S. Este proceso es conocido como **multiprogramación** o **multitarea**.

Estos sistemas mantienen varios trabajos listos para ejecutarse en memoria principal, por lo que requieren alguna forma de gestión de memoria, además debe decidir cuál ejecutar, haciendo uso de un algoritmo de planificación.

Sistemas de tiempo compartido

La técnica de tiempo compartido consiste en que el tiempo del procesador se comparte entre diversos usuarios. Múltiples usuarios acceden simultáneamente al sistema por medio de terminales, donde el sistema operativo intercala la ejecución de cada programa de usuario en ráfagas cortas o cuantos (*quantum*) de computación. El tiempo de respuesta de un sistema correctamente diseñado debería ser comparable al de un computador dedicado.

Características de los sistemas operativos modernos

- ❖ Arquitectura micronúcleo.
- ❖ Multihilos.
- ❖ Multiproceso simétrico.
- ❖ Sistemas operativos distribuidos.
- ❖ Diseño orientado a objetos.

Arquitectura micronúcleo

La arquitectura micronúcleo asigna solamente unas pocas funciones esenciales al núcleo. Otros servicios del sistema operativo los proporcionan procesos que se ejecutan en modo usuario y que el micronúcleo trata como cualquier otra aplicación.

Este enfoque simplifica la implementación, proporciona flexibilidad, y se adapta bien para entornos distribuidos.

Multihilos

Los multihilos son una técnica por lo cual un proceso, ejecutando una aplicación, se divide en hilos que pueden ejecutarse concurrentemente.

- **Hilo:** unidad de trabajo que se puede expedir para su ejecución. Conjunto de instrucciones que pertenecen a un proceso. Se les suele llamar procesos ligeros o contextos de ejecución. Se ejecuta secuencialmente y es interrumpible para que el procesador puede ceder el turno a otro hilo.
- **Proceso:** un conjunto de uno o más hilos y los recursos del sistema asociados (código, datos, archivos abiertos y dispositivos). Al dividir una aplicación en múltiples hilos, el programador tiene un gran control sobre la modularidad de la aplicación y la coordinación de los sucesos relativos al sistema.

Los multihilos son muy útiles para las aplicaciones que ejecutan un número de tareas esencialmente independientes que no necesitan ser consecutivas.

Multiproceso simétrico

Es un término que se refiere a una arquitectura hardware y al comportamiento de la misma. Un multiprocesador simétrico puede definirse como un sistema de computadores de un solo usuario con las siguientes características:

- Existencia de múltiples procesadores.
- Estos procesadores comparten la misma memoria principal y dispositivos de E/S.
- Todos los procesadores pueden ejecutar las mismas funciones (simétrico).

Ventajas:

- **Rendimiento:** se puede ejecutar más de un proceso simultáneamente, cada uno en un procesador diferente.
- **Disponibilidad:** el fallo de un procesador no detiene la máquina, el sistema continúa funcionando con un rendimiento reducido.
- **Crecimiento incremental:** se incrementa el rendimiento del sistema agregando un procesador adicional.
- **Escalabilidad:** variedad de productos con diferentes precios y características de rendimiento basados en el número de procesadores del sistema.

La existencia de múltiples microprocesadores es transparente al usuario.

Sistema operativo distribuido

Un sistema operativo distribuido proporciona la ilusión de un único espacio de memoria principal y un único espacio de memoria secundaria, además de otros mecanismos de acceso unificados.

Diseño orientado a objetos

Posibilita añadir extensiones modulares a un pequeño núcleo. Permite a los programadores personalizar el SO sin romper la integridad del mismo, y también facilita el desarrollo de herramientas distribuidas y sistemas operativos distribuidos abiertos.

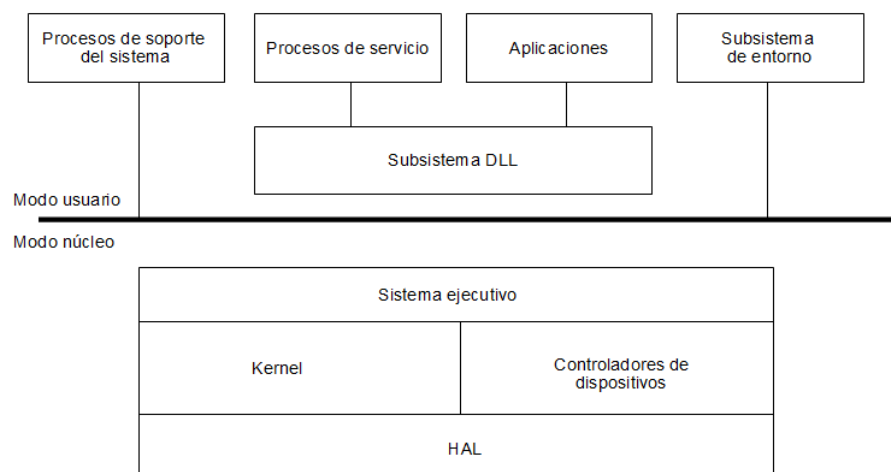
Microsoft Windows

Reseña histórica

- MS DOS 1.0 (1981). Sistema operativo de Microsoft para la primera computadora personal de IBM. Se ejecutaba en 8K de memoria utilizando el microprocesador Intel 8086.
- MS DOS 2.0 (1983). Para el PC XT de IBM. Añadió soporte para discos duros y ofrecía directorios jerárquicos (directorios con subdirectorios y archivos). Se añadieron también características de UNIX como el redireccionamiento de E/S (capacidad de cambiar la identidad de la entrada o salida de una aplicación) y la impresión subordinada (*background*). La memoria creció hasta 24 Kb.
- MS DOS 3.0 (1984). Para el PC AT de IBM con micro Intel 80286 que añadió direccionamiento extendido y recursos de protección de memoria, pero DOS no los utilizó. Uso el 80286 como un “8086 rápido” para mantener compatibilidad. Los requisitos de memoria crecieron hasta 36 KB.
 1. MS DOS 3.1 (1984). Añade soporte para redes de PC.
 2. MS DOS 3.3 (1987). Soporte para PS/2. El sistema pasó a utilizar 46 KB.
- Windows 3.0 (1990). El DOS estaba siendo utilizado en un entorno que iba más allá de sus capacidades. Microsoft comienza el desarrollo de una Interfaz Gráfica de Usuario (GUI) que podría ejecutarse entre el usuario y el DOS. En 1990, Microsoft presenta una versión de la GUI, el Windows 3.0, la cual seguía maniatada por la necesidad de ejecutarse sobre DOS.

- Windows NT (1993). Luego de una tentativa fallida por Microsoft de desarrollar junto a IBM un SO de nueva generación que aprovechara las características de los actuales microprocesadores, IBM desarrolla OS/2 y Microsoft lanza Windows NT. Ofrece multitarea en entornos monousuarios y multiusuarios.
- Windows NT 4.0. Después de varias versiones NT 3.X, Microsoft desarrolla la versión 4.0, que proporciona la misma interfaz de Windows 98. El mayor cambio consiste en que varios de los componentes gráficos que se ejecutaban tanto en modo usuario como formando parte del subsistema Win 32 en 3.X, han sido trasladados al ejecutor de Windows NT, que ejecuta en modo núcleo, lo que implica una aceleración en tales ejecuciones. El inconveniente es que servicios del sistema tienen acceso a funciones graficas de bajo nivel, pudiendo impactar con la fiabilidad del SO.
- Windows 2000 (2000). Nuevamente el ejecutor y la arquitectura micronúcleo son fundamentalmente los mismos que en NT 4.0. Se han añadido nuevas características, como el soporte al procesamiento distribuido. El directorio activo es un servicio del directorio distribuido capaz de traducir los nombres de objetos arbitrarios a cualquier tipo de información acerca de los mismos. Una consideración final sobre Windows 2000 es la distribución entre Windows 2000 Server y Professional, en donde la versión Server incluye servicios para su uso como servidor de red.
- Windows XP (2001).
- Windows Server 2003 (2003).
- Windows Vista (2007).
- Windows Server 2008 (2008).
- Windows 7 (2009).
- Windows 8 (2011).
 1. Windows 8.1 (2012).
- Windows Server 2012 (2012).

Arquitectura



Como en casi todos los sistemas operativos, se separa el software orientado a las aplicaciones del software del sistema operativo.

El software del modo núcleo tiene acceso a los datos del sistema y al hardware. El software restante, que se encuentra en modo usuario, tiene acceso limitado a los datos del sistema.

Organización del sistema operativo

Windows no tiene una arquitectura micronúcleo pura, sino una modificación de la misma.

Windows es altamente modular. Cualquier modulo puede ser eliminado, actualizado o reemplazado sin volver a escribir el sistema completo o sus Interfaces de Programas de Aplicación (API). Sin embargo, a diferencia de un sistema micronúcleo, Windows está configurado para que la mayoría de las funciones externas al micronúcleo se ejecuten en modo núcleo, todo por una razón de rendimiento.

Uno de los objetivos de diseño de Windows es la compatibilidad, que sea capaz de ejecutar sobre distintas plataformas de hardware, por eso utiliza la siguiente estructura por niveles:

- **Capa de abstracción del hardware (HAL):** aísla el sistema operativo de las diferencias de hardware específicas de cada plataforma. El SO da órdenes que pasan por la HAL y ésta las “transforma” para que el hardware específico las entienda; y al revés, el hardware da órdenes a al SO a través de la HAL para que el SO las entienda. Así no hay dependencia entre el SO y el hardware.
- **Núcleo (kernel):** formado por los componentes más usados y fundamentales del sistema operativo (planificación de hilos, cambios de contexto, gestión de excepciones e interrupciones y sincronización de microprocesadores).
- **Controladores de dispositivos:** incluye los *drivers* de dispositivos que traducen las llamadas a funciones de E/S del usuario a peticiones a dispositivos hardware de E/S específicos.
- **Sistema ejecutivo:** Contiene los servicios básicos del sistema operativo, como la gestión de memoria, la gestión de procesos e hilos, seguridad, E/S y comunicación entre procesos.

Módulos del Sistema ejecutivo

El Sistema Ejecutivo de Windows incluye módulos para las funciones específicas del sistema y proporciona una API para el software en modo usuario.

- **Administrador de E/S:** distribuye los controladores de dispositivos (*drivers*) entre los distintos procesos.
- **Administrador de objetos:** crea, gestiona y elimina los objetos del Ejecutor de Windows.
- **Monitor de seguridad:** impone la validación de acceso y la generación de reglas de auditoría para los objetos protegidos.
- **Administrador de procesos/hilos:** crea y elimina objetos y sigue la pista de los objetos proceso e hilo.
- **Servicios de llamadas a procedimiento local (LPC):** impone las relaciones cliente/servidor entre las aplicaciones y los subsistemas del Ejecutor dentro de un mismo sistema, de forma similar al servicio de llamadas a procedimiento remoto (RPC) utilizadas en el procesamiento distribuido.
- **Administrador de memoria virtual:** traduce direcciones virtuales en el espacio de direcciones del proceso a páginas físicas en la memoria de la computadora.
- **Administrador de caché:** mejora el rendimiento de las E/S sobre archivos.
- **Módulos para gráficos/ventanas:** crea las interfaces de pantalla (ventanas) y gestiona los dispositivos gráficos.

Procesos de usuario

- **Procesos de soporte de sistemas especiales:** proceso de inicio de sesión y la gestión de sesión.
- **Procesos de servicio:** otros servicios como el registro de sucesos.
- **Subsistemas de entorno:** ofrecen servicios nativos de Windows a las aplicaciones de usuario y de este modo proporcionan un entorno de SO. Cada subsistema incluye DLLs (bibliotecas de enlace dinámico) que convierten las llamadas de las aplicaciones de usuario en llamadas que interprete Windows.
- **Aplicaciones de usuario:** programas que se ejecutan sobre el SO. Pueden ser Win32, Posix, OS/2, MS-DOS, etc.

Modelo cliente/servidor

Cada subsistema de entorno y subsistema del servicio ejecutivo se implementan como uno o más procesos. Cada proceso espera la solicitud de un cliente para uno de sus servicios. Un cliente (un programa de aplicación u otro módulo del SO) solicita un servicio enviando un mensaje. El mensaje es encaminado a través del sistema ejecutivo hasta el servidor apropiado. El servidor lleva a cabo la operación solicitada y devuelve el resultado o la información de estado por medio de otro mensaje, que se encamina de vuelta al cliente mediante el servicio ejecutivo.

Ventajas

- Simplifica el sistema ejecutivo. Es posible construir diversos API. Se pueden añadir fácilmente nuevas interfaces.
- Mejora la fiabilidad. Cada módulo de los servicios ejecutivos se ejecuta como un proceso separado, con su propia partición de memoria, protegido de otros módulos. Un cliente puede fallar sin corromper el resto del SO.
- Proporciona una base adecuada para la computación distribuida.

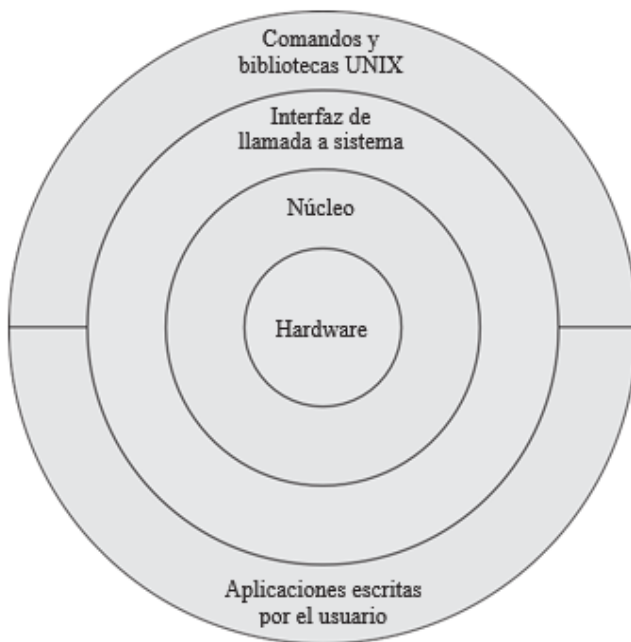
UNIX

Historia

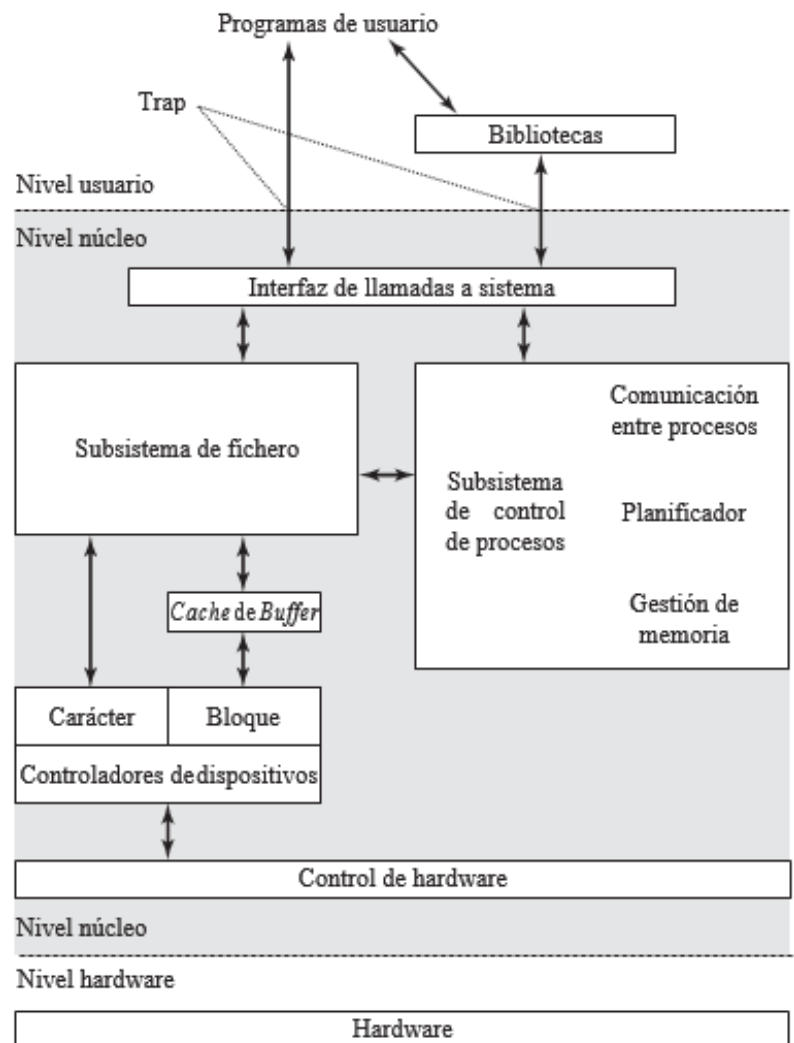
- Se desarrolló en los Laboratorios Bell operando en una PDP-7 en 1970.
- Se reescribió en lenguaje de programación C, demostrando las ventajas de usar un lenguaje de alto nivel.

Descripción

El hardware básico está rodeado por el software del SO (el *kernel*). UNIX cuenta con varios servicios e interfaces de usuario que se consideran parte del sistema (interprete de órdenes, conocido como *shell*, o un software de interfaz gráfica). El nivel exterior está formado por las aplicaciones de los usuarios y la interfaz de usuario del compilador C. Los programas de usuario pueden invocar servicios del SO directamente o a través de programas de biblioteca (librerías).



Arquitectura general de UNIX



Núcleo tradicional de UNIX

Linux

Historia

- Linux aparece como variante de UNIX para la arquitectura del IBM PC.
- La versión inicial fue escrita por Linus Torvals.
- Torvals distribuyó una primera versión de Linux en Internet en 1991. Desde entonces un gran número de personas han contribuido al desarrollo de Linux.
- Linux es libre y está disponible su código fuente.
- Linux es altamente modular y fácilmente configurable. Esto hace que sea fácil de obtener el rendimiento óptimo para múltiples plataformas de hardware.

Estructura modular

Linux está organizado como un conjunto de bloques independientes denominados módulos cargables, que tienen dos características importantes:

- **Enlace dinámico:** un módulo del núcleo puede cargarse y enlazarse dentro del núcleo. También, un módulo puede ser desenlazado y borrado de la memoria en cualquier momento. Esto facilita la tarea de configuración y protege la memoria del núcleo. En Linux, un programa de usuario o un usuario puede cargar y descargar explícitamente módulos del núcleo usando las órdenes *insmod* y *rmmod*.
- **Módulos apilables:** los módulos se organizan jerárquicamente, pudiendo definir dependencias entre módulos. Esto tiene dos ventajas:
 1. El código común de un conjunto de módulos similares puede ser trasladado a un único módulo, reduciendo la duplicación.
 2. El núcleo puede asegurar que los módulos necesarios estén presentes, despreocupándose de descargar un módulo del que dependen otros módulos en ejecución y cargando cualquier módulo adicional necesario cuando se cargue un módulo nuevo.

Los principales componentes del núcleo son los siguientes:

•**Señales.** El núcleo utiliza las señales para llamar a un proceso. Por ejemplo, las señales se utilizan para notificar ciertos fallos a un proceso como, por ejemplo, la división por cero.

•**Llamadas al sistema.** Forma en la cual un proceso requiere un servicio de núcleo específico.

•**Procesos y planificador.** Crea, gestiona y planifica procesos.

•**Memoria virtual.** Asigna y gestiona la memoria virtual para los procesos.

•**Sistemas de ficheros.** Proporciona un espacio de nombres global y jerárquico para los ficheros, directorios y otros objetos relacionados con los ficheros. Además, proporciona las funciones del sistema de ficheros.

•**Protocolos de red.** Da soporte a la interfaz Socket para los usuarios, utilizando la pila de protocolos TCP/IP.

•**Controladores de dispositivo tipo carácter.** Gestiona los dispositivos que requiere el núcleo para enviar o recibir datos un byte cada vez, como los terminales, los módems y las impresoras.

•**Controladores de dispositivo tipo bloque.** Gestiona dispositivos que leen y escriben datos en bloques, tal como varias formas de memoria secundaria (discos magnéticos, CDROM, etc.).

•**Controladores de dispositivo de red.** Gestiona las tarjetas de interfaz de red y los puertos de comunicación que permiten las conexiones a la red, tal como los puentes y los encaminadores.

•**Traps y fallos.** Gestiona los traps y fallos generados por la CPU, como los fallos de memoria.

•**Memoria física.** Gestiona el conjunto de marcos de páginas de memoria real y asigna las páginas de memoria virtual.

•**Interrupciones.** Gestiona las interrupciones de los dispositivos periféricos.

Unidad N°2 – Administración y gestión de archivos

Archivos

Archivo: Son unidades lógicas de información creada por los procesos. Espacio de direcciones lógicas contiguas. Son un mecanismo de abstracción.

Nombre

Cuando un proceso crea un archivo, le asigna nombre. Cuando el proceso termina, el archivo sigue existiendo y otros procesos pueden tener acceso a él mediante su nombre.

Las reglas exactas para nombrar archivos varían según el sistema. Algunos sistemas de archivos distinguen entre mayúsculas y minúsculas (UNIX), pero otros no (MS-DOS).

Muchos sistemas de archivos manejan nombres de archivos en dos partes, separadas con un punto. La parte que sigue al punto se denomina **extensión de archivo**, y por lo general indica algo acerca del archivo. En algunos sistemas (como UNIX) las extensiones de archivo son sólo convenciones y el SO no vigila que se usen de alguna forma específica. El nombre sirve de recordatorio para el usuario en vez de servirle información a la computadora. En cambio, Windows tiene conocimiento de las extensiones y les asigna un significado. A cada extensión se le asigna un programa “dueño”, haciendo que cuando un usuario haga doble clic sobre el archivo, se abra el programa asociado a la extensión con el nombre de archivo como parámetro (*si haces doble clic en documento.doc se abre documento.doc en Word*).

Estructuras de archivos

1. **Sucesión no estructurada de bytes:** el SO no sabe que contiene el archivo, ni le interesa; lo único que ve son bytes. Cualquier significado debe ser impuesto por los programas a nivel usuario. Utilizado por UNIX y Windows. Ofrece la máxima flexibilidad al considerar los archivos solo como secuencias de bytes.
2. **Secuencia de registros de longitud fija:** cada registro tiene cierta estructura interna. La operación de lectura devuelve un registro y la operación de escritura sobrescribe o anexa un registro.
3. **Árbol de registros:** los registros pueden ser de diferente longitud. Cada registro posee un campo clave en una posición fija del registro. El árbol se ordena según el campo clave con el objetivo de encontrar con rapidez una clave en particular.

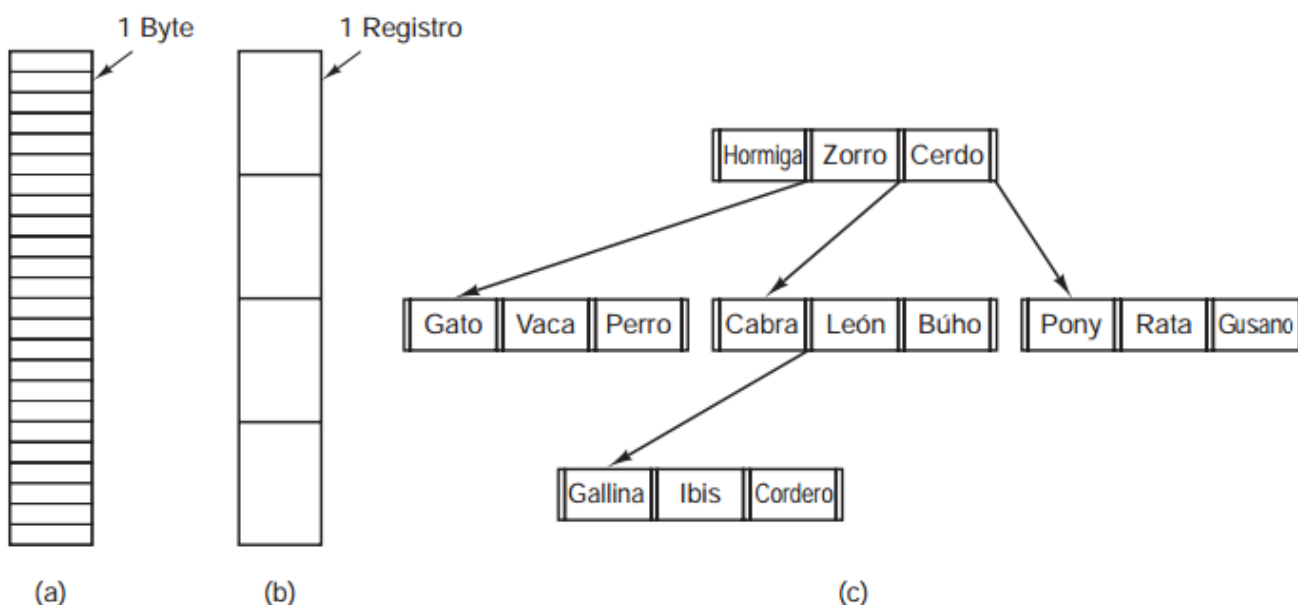


Figura 4-2. Tres tipos de archivos. (a) Secuencia de bytes. (b) Secuencia de registros. (c) Árbol.

Tipos de archivos

1. **Archivos regulares:** contienen información del usuario. Figura 4-2. Pueden ser de texto (ASCII) o binarios.
 - a. ASCII: consisten en líneas de texto. Pueden exhibirse e imprimirse tal cual, y pueden editarse con cualquier editor de textos. Facilitan la conexión de la salida de un programa con la entrada de otro.
 - b. Binarios: poseen una estructura interna conocida por los programas que los usan. Solo pueden ser visualizados por la aplicación que los creó.
2. **Directorios:** archivos del sistema que sirven para mantener la estructura del sistema de archivos (contiene otros archivos y directorios).
3. **Archivos especiales de caracteres:** tienen que ver con E/S y sirven para modelar dispositivos de E/S en serie, tales como terminales, impresoras y redes.
4. **Archivos especiales de bloques:** sirven para modelar discos (se leen y escriben bloques).

Métodos de acceso

- **Acceso secuencial:** el archivo se lee desde el principio hasta el final, no pudiendo realizar saltos o leerlos en otro orden. El tiempo de acceso es lento (excepto si el archivo se encuentra en el principio). Usado en cintas magnéticas.
- **Acceso aleatorio:** los archivos pueden leerse en cualquier orden. Se puede acceder a ellos a través de una clave o índice independientemente de su posición, sin tener que pasar por los datos anteriores. Usado en discos.

Atributos de un archivo

Atributo: información adicional de un archivo que agrega el SO, como la fecha y la hora de creación, tamaño, etc. Los atributos varían según el SO.

- | | |
|---|--|
| • <u>Protección</u> : quien puede tener acceso al archivo y cómo. | • <u>Indicador temporal</u> : 0 normal; 1 borrarlo al terminar el proceso. |
| • <u>Contraseña</u> : clave necesaria para poder acceder al archivo. | • <u>Indicador de bloqueo</u> : 0 sin bloqueo; 1 bloqueado. |
| • <u>Creador</u> : ID del creador del archivo. | • <u>Longitud de registro</u> : bytes en un registro. |
| • <u>Propietario</u> : propietario actual. | • <u>Posición de clave</u> : distancia a la clave dentro de cada registro. |
| • <u>Indicador de sólo lectura</u> : 0 para leer/escribir; 1 sólo lectura. | • <u>Longitud de clave</u> : bytes en el campo clave. |
| • <u>Indicador de oculto</u> : 0 normal; 1 para no mostrar. | • <u>Hora de creación</u> : fecha y hora de creación. |
| • <u>Indicador de sistema</u> : 0 normal; 1 archivo de sistema. | • <u>Hora de último acceso</u> : fecha y hora en se tuvo acceso por última vez al archivo. |
| • <u>Indicador de archivado</u> : 0 ya respaldado; 1 debe respaldarse. | • <u>Hora de último cambio</u> : fecha y hora en se modificó por última vez el archivo. |
| • <u>Indicador de ASCII/Binario</u> : 0 ASCII; 1 binario. | • <u>Tamaño actual</u> : bytes en el archivo. |
| • <u>Indicador de acceso aleatorio</u> : 0 acceso secuencial; 1 acceso aleatorio. | • <u>Tamaño máximo</u> : bytes que puede alcanzar el archivo. |

Operaciones con archivos

Los SO ofrecen diferentes operaciones para almacenar y recuperar la información. Las más comunes son:

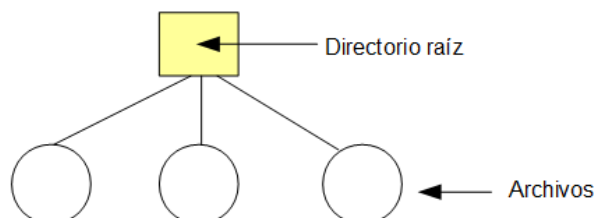
1. **CREATE (crear):** se crea el archivo sin datos. Se reserva espacio en el disco y se establecen algunos atributos.
2. **DELETE (borrar):** borrar un archivo para liberar espacio en disco.
3. **OPEN (abrir):** hace que el sistema obtenga los atributos y la lista de direcciones de disco y los coloque en la memoria principal para después accederlos
4. **CLOSE (cerrar):** cierra el archivo cuando han acabado todos los accesos, ya que no son necesarios los atributos y direcciones en disco.
5. **READ (leer):** se leen datos del archivo. Se lee desde la posición actual, y debe especificarse cuantos datos se necesita y el búfer donde deben colocarse.
6. **WRITE (escribir):** lee los datos de un archivo, generalmente, en la posición actual. Si la posición actual es el fin del archivo, aumenta el tamaño; si la posición actual es un punto intermedio, se sobrescriben los datos.
7. **APPEND (añadir):** forma restringida de WRITE; solo se pueden añadir datos al final del archivo.
8. **SEEK (buscar):** para los archivos de acceso aleatorio, especifica el punto a partir del cual se tomaran los datos (escribir o leer).
9. **GET ATTRIBUTES:** se leen los atributos de un archivo para que un proceso los utilice.
10. **SET ATTRIBUTES:** el usuario puede establecer algunos atributos o modificarlos después de la creación del archivo.
11. **RENAME (renombrar):** cambiar el nombre de un archivo existente.

Directorios

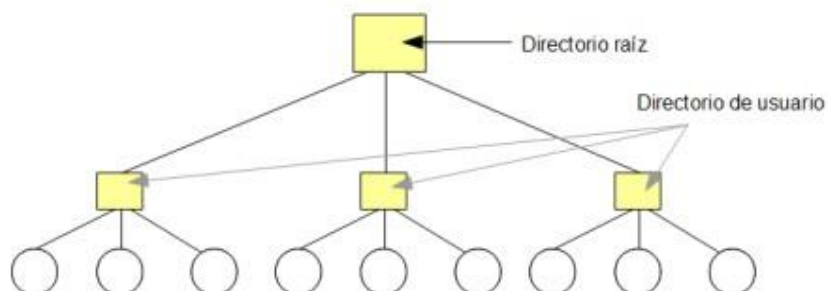
Directorio: son archivos que contienen a otros archivos o directorios. Son carpetas.

Jerarquía de directorios

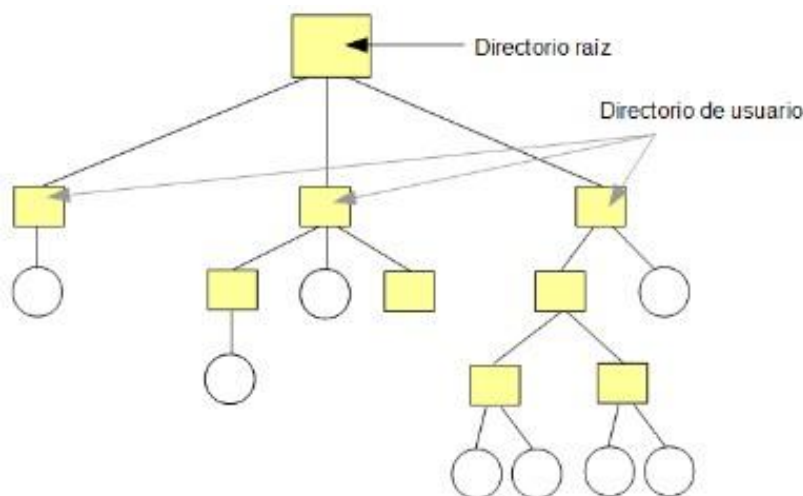
- **Directorio Raíz:** directorio que contiene a todos los archivos.
- **Directorios de un solo nivel:** un directorio (directorio raíz) contiene todos los archivos. Las ventajas son la sencillez y la rapidez para buscar archivos. Este sistema es problemático en sistemas multiusuario (ejemplo, dos usuarios crean un archivo con el mismo nombre sobrescribiendo al anterior).



- **Directorios de dos niveles:** un directorio raíz que contiene a su vez un directorio para cada usuario, eliminando los conflictos presentes en el sistema de directorios de un nivel. No satisface a los usuarios que tienen una gran cantidad de archivos.



- **Directorios jerárquicos:** consiste en una jerarquía general, un árbol de directorios. Posibilita crear un número arbitrario de subdirectorios ofreciendo a los usuarios una herramienta potente para organizar sus archivos.



Nombres de ruta

Ruta: son mecanismos para especificar nombres de archivos en un árbol de directorios.

- **Ruta absoluta:** consiste en el camino que hay que seguir para llegar desde el directorio raíz hasta el archivo.
- **Ruta relativa:** consiste en el camino que hay que seguir desde el directorio de trabajo (directorio actual) hasta el archivo.

Un usuario puede designar un directorio como el directorio de trabajo actual, en cuyo caso todos los nombres de las rutas que no empiecen en el directorio raíz se toman en forma relativa al directorio de trabajo.

Operaciones con directorios

Ejemplos de operaciones de UNIX:

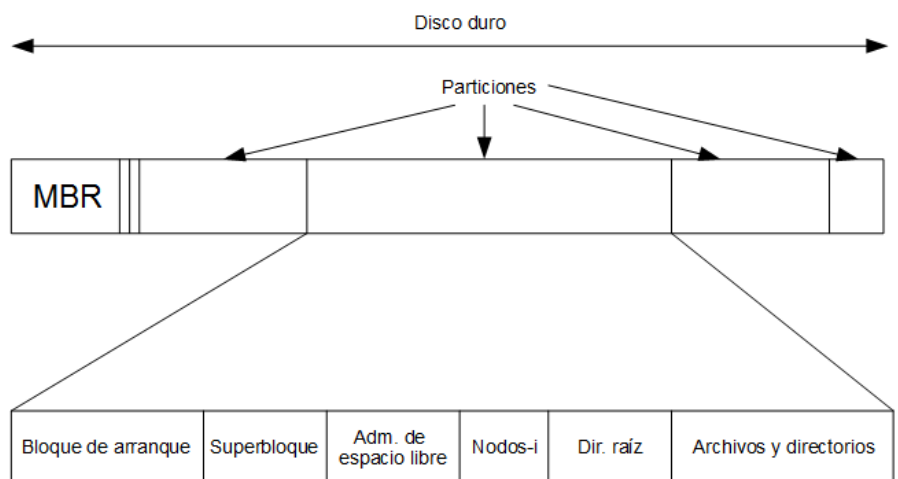
1. **CREATE (crear):** crea un directorio vacío, excepto por punto y punto-punto que se crean automáticamente por el sistema o en unos cuantos casos lo hace el programa mkdir.
2. **DELETE (borrar):** elimina un directorio, siempre y cuando esté vacío (no se considera punto ni punto-punto).
3. **OPENDIR (abrir):** Los directorios se pueden leer. Por ejemplo, para listar todos los archivos en un directorio, un programa de listado abre el directorio para leer los nombres de todos los archivos que contiene. Antes de poder leer un directorio se debe abrir, en forma análoga al proceso de abrir y leer un archivo.
4. **CLOSEDIR (cerrar):** cierra un directorio para desocupar espacio en las tablas internas.
5. **READDIR (leer):** devuelve la siguiente entrada en un directorio abierto. Siempre devuelve una entrada en formato estándar, sin importar la estructura de directorio.
6. **RENAME (renombrar):** cambiar el nombre del directorio.
7. **LINK (enlazar):** permite que un archivo aparezca en más de un directorio. A un vínculo de este tipo, que incrementa el contador en el nodo-i del archivo se le llama algunas veces vínculo duro (o liga dura), una variante es el vínculo simbólico.
8. **UNLINK (desenlazar):** se elimina una entrada de directorio. Si el archivo que se quiere eliminar aparece solo en un directorio (caso normal), se elimina del sistema de archivos. Si está presente en varios directorios, solo se elimina la ruta de acceso especificada, los demás permanecen.

Implementación de sistemas de archivos

Organización del sistema de archivos

Los sistemas de archivos se almacenan en discos. Los discos pueden dividirse en particiones, con sistemas de archivos independientes en cada partición. El sector 0 del disco se llama registro maestro de arranque (**MBR**, *Master Boot Record*) y sirve para arrancar la computadora. Al final del MBR está la tabla de particiones, que contiene las direcciones inicial y final de cada partición. Una partición estará marcada como activa. Cuando se enciende la computadora, el BIOS lee el MBR del disco y lo ejecuta. Lo primero que hace el programa del MBR es localizar la partición activa, leer su primer bloque, llamado **bloque de arranque**, y ejecutarlo. El programa del bloque de arranque carga el SO en esa partición.

La organización de una partición varía según el sistema de archivos. El primer elemento es el **superbloque** que contiene todos los parámetros clave acerca del sistema de archivos y se transfiere del disco a la memoria cuando se arranca la computadora. A continuación, podría haber información acerca de bloques libres, luego podrían estar los nodos-i. Después podría venir el directorio raíz, y por último el resto del disco suele contener los demás directorios y archivos.



Implantación de archivos

- **Asignación continua:** es el esquema de asignación más simple que consiste en almacenar cada archivo en una serie contigua de bloques de disco. Usado en los CD y DVD.

Ventajas:

- Fácil implementación: para localizar los bloques de un archivo solo se necesitan dos números, la dirección del primer bloque y el número de bloques del archivo.
- Buen desempeño en lectura: puede leerse todo el archivo del disco en una sola operación. Sólo necesita un solo desplazamiento del brazo del disco (para el primer bloque).

Desventajas:

- Fragmentación del disco: cuando se elimina un archivo, sus bloques se liberan, dejando una serie de bloques libres en el disco, resultado que el disco se componga de archivos y huecos.
- Cuando se va a crear un archivo nuevo se hace necesario conocer su tamaño final para recoger un hueco del tamaño correcto en el cual colocarlo. Este diseño es molesto para el usuario que no sabe cuál va a ser el tamaño del archivo.

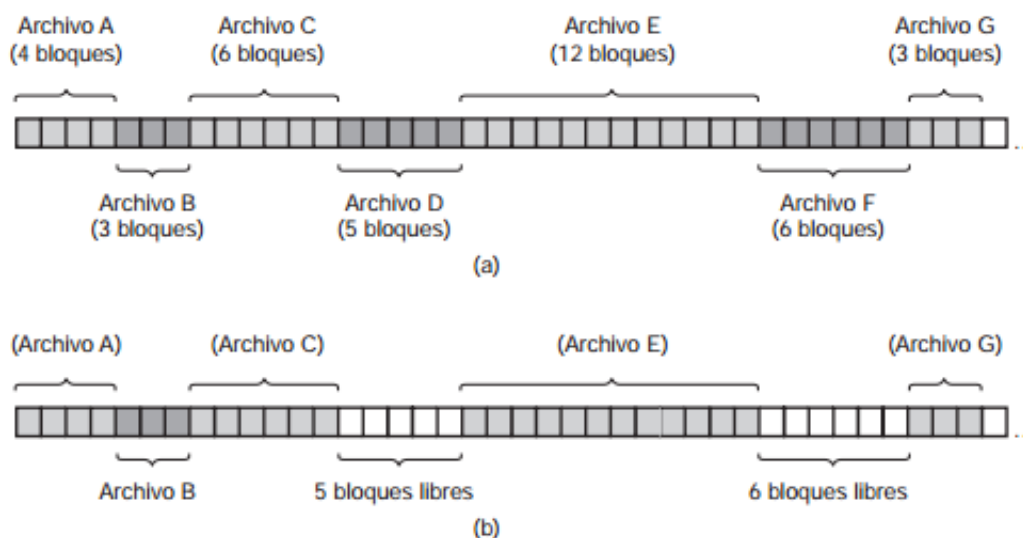


Figura 4-10. (a) Asignación contigua de espacio de disco para siete archivos. (b) El estado del disco después de haber removido los archivos D y F.

- **Asignación por lista enlazada:** consiste en mantener cada archivo como una lista enlazada de bloques de disco. La primera palabra de cada bloque se usa como puntero al siguiente bloque, el resto del bloque es para datos.

Ventajas:

- Pueden usarse todos los bloques del disco. No se pierde espacio por fragmentación del disco.
- Sólo es necesario conocer la dirección del primer bloque.

Desventajas:

- Acceso aleatorio lento. Para llegar al bloque n , el SO tiene que comenzar por el principio y leer los $n-1$ bloques que lo preceden, uno por uno.
- La cantidad de datos almacenados en un bloque no es potencia de 2 porque el puntero ocupa unos cuantos bytes. Infiere en la eficiencia de programas que leen y escriben en bloques de tamaño de potencia de 2.

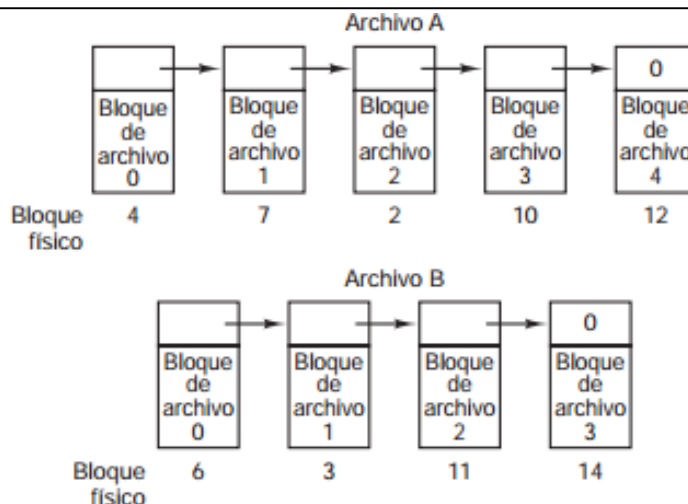


Figura 4-11. Almacenamiento de un archivo como una lista enlazada de bloques de disco.

- **Asignación por lista enlazada utilizando una tabla en la memoria:** se eliminan las desventajas de la asignación por lista enlazada ya que se saca el apuntador de cada bloque de disco y se lo coloca en una tabla en la memoria denominada “tabla de asignación de archivo” (FAT, *file allocation table*).

Ventajas:

- Todo el bloque se utiliza para datos.
- Acceso aleatorio mucho más fácil.
- La cadena está cargada en memoria, pudiéndose seguir sin tener que leer el disco.
- Se necesita la dirección del primer bloque para localizar todos los demás.

Desventajas:

- Toda la tabla debe estar en memoria todo el tiempo.
- No se escala muy bien en los discos grandes.

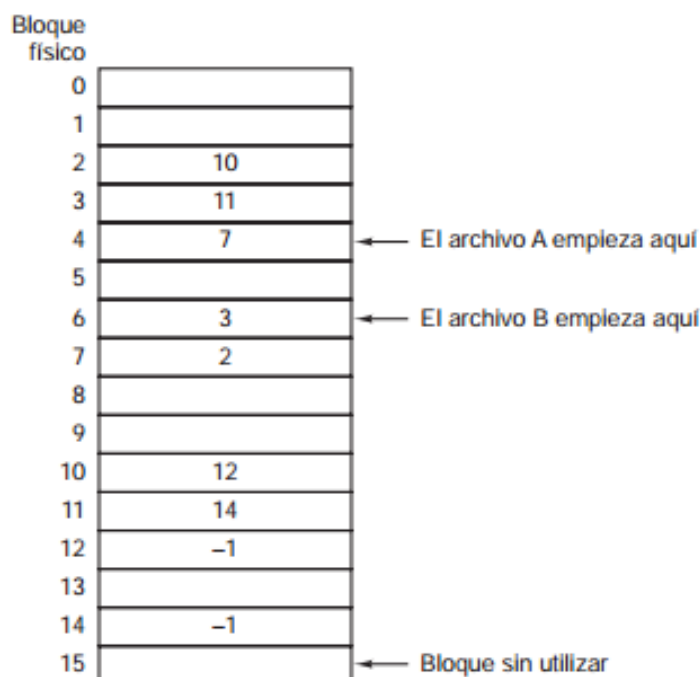
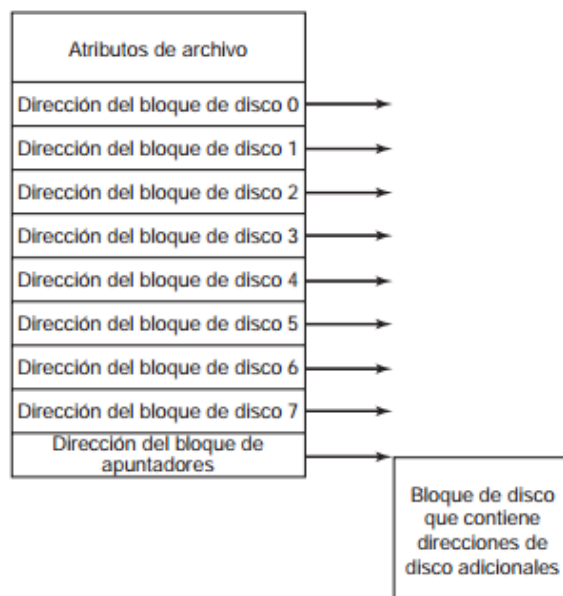


Figura 4-12. Asignación de lista enlazada que utiliza una tabla de asignación de archivos en la memoria principal.

- **Nodos-i:** asocia a cada archivo una estructura de datos llamada nodo-i (nodo índice) que contiene los atributos y direcciones de disco de los bloques del archivo. Dado el nodo-i, es posible hallar todos los bloques del archivo. Es usado en Linux.



Por lo general, este arreglo es mucho más pequeño que el espacio ocupado por la tabla de archivos descrita en la sección anterior. La razón es simple: la tabla para contener la lista enlazada de todos los bloques de disco es proporcional en tamaño al disco en sí. Si el disco tiene n bloques, la tabla necesita n entradas. A medida que aumenta el tamaño de los discos, esta tabla aumenta linealmente con ellos. En contraste, el esquema del nodo-i requiere un arreglo en memoria cuyo tamaño sea proporcional al número máximo de archivos que pueden estar abiertos a la vez. No importa si el disco es de 10 GB, de 100 GB o de 1000 GB.

Ventaja:

- El nodo-i sólo está en memoria cuando el archivo correspondiente está abierto.

Desventaja:

- Cada nodo-i tiene espacio para un número fijo de dirección de disco, haciendo que se llene cuando el archivo crece. Para solucionar esto se reserva la última dirección de disco no para un bloque de datos, sino para dirección de un bloque que contenga más direcciones de bloques de disco. Para archivos más grandes, una de las direcciones en el nodo-i es la dirección de un bloque en el disco llamado **bloque simplemente indirecto**, el cual contiene direcciones en disco adicionales. Si con esto no es suficiente, otra dirección del nodo-i, el **bloque doblemente indirecto** contiene la dirección de un bloque que tiene una lista de bloques simplemente indirectos, donde cada uno de estos apunta a unos centenares de bloques de datos. Si aún esto no basta, se puede utilizar un **bloque triplemente indirecto**.

El inode mismo contiene 15 entradas de dirección de bloques del largo de la arquitectura (ej: 32 bits) que apuntan a bloques con datos:

- Los primeros 12 bloques referencian directamente bloques donde están los datos en disco

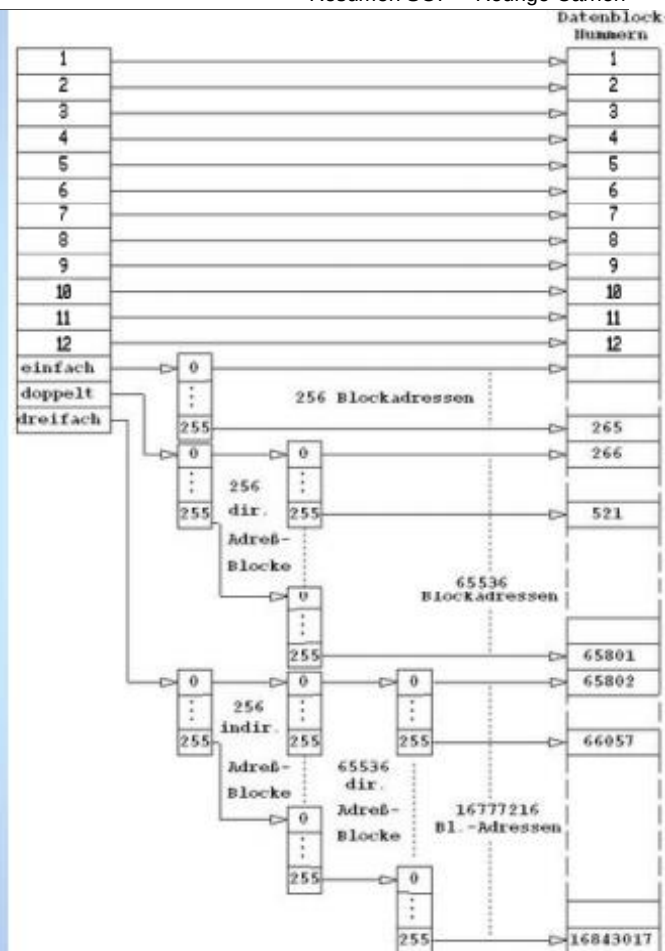
- Los 3 últimos son *punteros indirectos* que permiten agregar mas direcciones de bloque al archivo:

- 1 puntero a bloques indirectos: la dirección de un bloque que contiene tantos punteros a bloques de datos como quepan en un bloque de disco.

- 1 puntero a bloques indirectos dobles: la dirección de un bloque, donde están direcciones de bloques que contienen direcciones de bloques de datos

- 1 puntero a bloques indirectos triples: ídem, con un nivel más

Dada esta estructura, se tienen los 16 millones de bloques direccionados, el tamaño máximo de los archivos esta dado por el tamaño de bloque dado en el formato del sistema de archivos.



Implantación de directorios

Para poder leer un archivo, es preciso abrirlo primero. Cuando se abre un archivo, el SO utiliza el nombre de ruta proporcionado por el usuario para localizar la entrada de directorio. Ésta proporciona la información necesaria para hallar los bloques de disco. Dependiendo del sistema, esta información puede ser la dirección de disco de todo el archivo (con asignación contigua), el número del primer bloque (ambos esquemas de lista enlazada) o el número del nodo-i. En todos los casos, la función principal del sistema de directorios es establecer una correspondencia entre el nombre de archivo ASCII y la información necesaria para localizar datos. Todo sistema de archivos mantiene atributos de los archivos.

Una posibilidad obvia es guardarlos directamente en la entrada de directorio. Muchos sistemas hacen precisamente esto. En un diseño sencillo, un directorio es una lista de entradas de tamaño fijo, una por archivo, que contiene un nombre de archivo (de longitud fija), una estructura con los atributos del archivo y una o más direcciones de disco que indiquen donde están los bloques de disco.

En los sistemas que usan Nodos-i, otra posibilidad es almacenar los atributos en los nodos-i, en lugar de en las entradas de directorio (la entrada de directorio es más corta: solo guarda nombre y nodo-i). Hasta ahora hemos supuesto que los archivos tienen nombres cortos de longitud fija. Sin embargo, casi todos los SO actuales reconocen nombres de archivos más largos, de longitud de variable. Una alternativa para esto, es abandonar la idea de que todas las entradas de directorio tienen el mismo tamaño. Con este método, cada entrada contiene una porción fija, que por lo regular principia con la longitud de la entrada y seguida de los atributos. Este encabezado de longitud fija va seguido del nombre de archivo en sí, que puede tener cualquier longitud. Una desventaja de este método es que cuando se elimina un archivo queda en el directorio un hueco de tamaño variable (mismo caso de la asignación continua), aunque es factible compactar el directorio porque está por completo en memoria. Otro problema es que una sola entrada de directorio podría cruzar fronteras de página, por lo que podría presentarse un fallo de página durante la lectura de un nombre de archivo.

Otra forma de manejar los nombres variables es hacer que todas las entradas de directorio sean de longitud fija y mantener los nombres de archivo juntos en un *heap* al final del directorio. Tiene la ventaja de que si se elimina una entrada siempre cabrá ahí el siguiente archivo creado.

Una pequeña ganancia aquí es que ya no hay una verdadera necesidad de que los nombres de archivos empiecen en límites de palabras, por lo que no se necesitan caracteres de relleno después de los nombres de archivos en la figura 4-15(b), como se hizo en la figura 4-15(a).

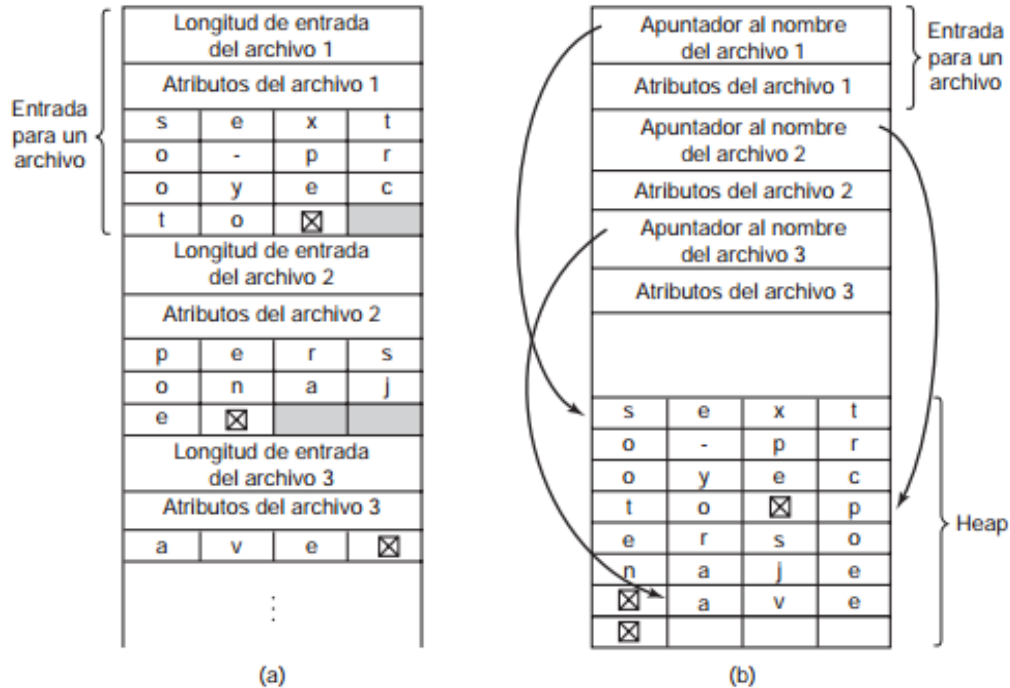
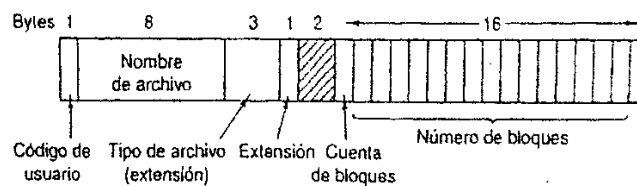


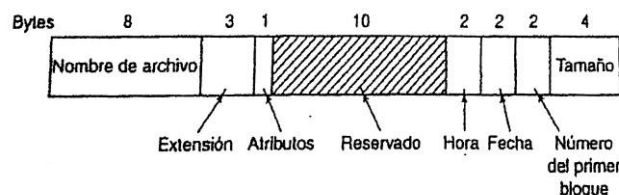
Figura 4-15. Dos maneras de manejar nombres de archivos largos en un directorio.
(a) En línea. (b) En un heap.

Una forma de acelerar la búsqueda es utilizar una tabla de hash en cada directorio. Se examina la entrada de tabla correspondiente al código de hash calculado a partir del nombre del archivo especificado. Si está desocupada, se coloca en ella un apuntador a la entrada del archivo. Si ya está en uso esa ranura, se construye una lista enlazada, encabezada por esa entrada de tabla, que encadena todas las entradas que tienen el mismo valor de hash. Aunque la búsqueda es más rápida, la administración de este método es más compleja.

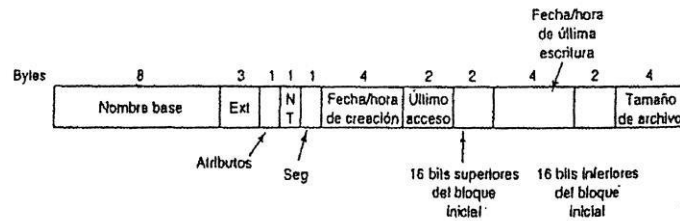
- **Directorios en CP/M:** sólo hay un directorio, lo único que el sistema de archivos tiene que hacer para consultar un nombre de archivo es buscarlo en el directorio. Una vez que encuentra la entrada, también tiene de inmediato los números de bloque en el disco, ya que están almacenados ahí mismo en la entrada, al igual que todos los atributos.



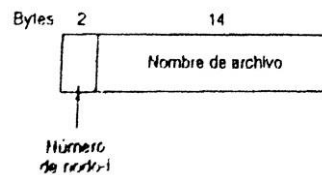
- **Directorios en MS-DOS:** los directorios tienen tamaño variable, sus entradas son de 32 bytes de longitud fija y contiene el nombre de archivo, los atributos y el número del primer bloque de disco, que se usa como índice de una tabla contenida en memoria (asignación por lista enlazada con una tabla en memoria). Siguiendo la cadena, se pueden encontrar todos los bloques.



- **Directorios en Windows 98:** nombres de archivo más largos y se introdujo FAT-32. Se adicionan cinco campos nuevos donde estaban los 10 bytes desocupados. El campo NT sirve para asegurar la compatibilidad con Windows NT. Los archivos poseían dos nombres, uno largo para Windows NT y un nombre de 8 + 3 por compatibilidad. Se puede acceder desde los dos nombres.

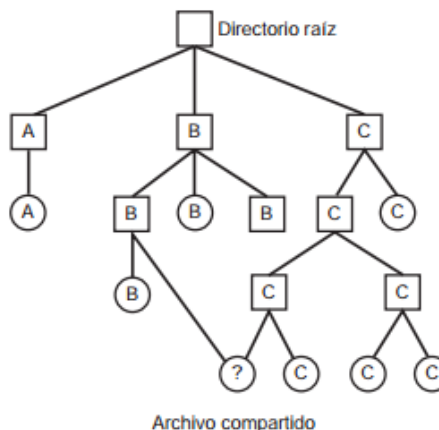


- **Directorios en UNIX:** cada entrada de directorio contiene sólo un nombre de archivo y su número de nodo-i. Toda la información acerca del tipo, tamaño, tiempos, propietario y bloques de disco está contenida en el nodo-i. Cuando se abre un archivo, el sistema de archivos debe tomar el nombre que se le proporciona y localizar sus bloques de disco. Lo primero que hace el sistema de archivos es localizar directorio raíz. En UNIX su nodo-i está situado en un lugar fijo del disco. La localización de un nodo-i una vez que se tiene su número es directa, ya que cada uno tiene una posición fija en el disco.



Archivos compartidos

Un archivo compartido es aquel que aparece al mismo tiempo en diferentes directorios que pertenecen a usuarios distintos. Si un archivo de un usuario A aparece en algún directorio de un usuario B, el archivo compartido se llama **enlace (link)**. El sistema de archivos deja de ser un árbol, es una gráfica acíclica dirigida.



Hay dos formas de enlazar archivos:

- **Enlace duro:** los directorios que comparten un archivo, no poseen los bloques de disco, sino que apuntan al mismo nodo-i que corresponde al archivo. Este es el esquema de UNIX.

Desventaja: si el propietario elimina el archivo y se limpia el nodo-i, algún directorio con un enlace a ese archivo, apuntará a un nodo-i no válido. Si el nodo-i se reasignara a otro archivo, el enlace apuntará a un archivo incorrecto. Una solución es eliminar el archivo original, pero mantener el nodo-i para el enlace, manteniendo la cuenta en 1.

- **Enlace simbólico (accesos directos):** el enlace se genera creando un archivo de tipo LINK en el directorio que quiere crear el enlace. Este archivo sólo contiene el nombre de ruta del archivo con el cual está enlazado, que es

utilizado para buscar el archivo original cuando se lo quiere leer.

Desventajas:

- Cuando el dueño elimina el archivo original, este se destruye, quedando los archivos de enlace obsoletos, ya que el sistema no lo va a poder localizar. Pasa algo similar cuando se mueve el archivo.
- Necesitan un procesamiento adicional: leer la ruta, analizarla, seguirla, componente por componente hasta llegar al nodo-i. Puede requerir muchos accesos a disco.
- Se necesita un nodo-i adicional por cada enlace, y si el nombre de ruta es largo, se necesita otro bloque adicional para almacenarla.

Ventaja: pueden servir para enlazar archivos por red, especificando la dirección de red de la máquina en la que está el archivo.

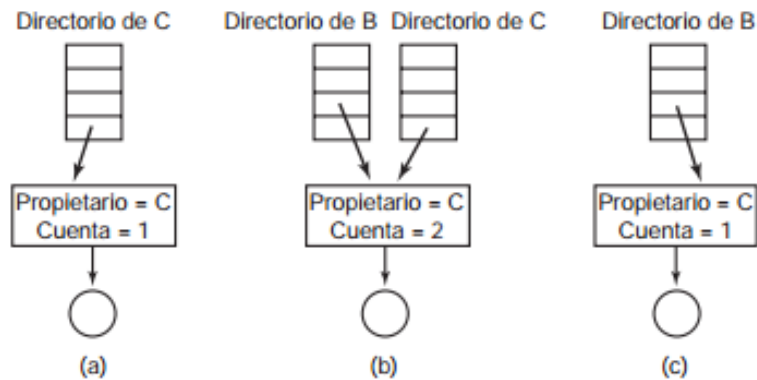


Figura 4-17. (a) Situación previa a la vinculación. (b) Después de crear el vínculo. (c) Después de que el propietario original elimina el archivo.

Implementación del Sistema de Archivos en Windows NTFS

NTFS o New Technology File System, es el sistema de archivos de la línea NT de los sistema MS Windows.

Diseñado desde cero, pensado para reemplazar a la familia de FAT eliminando todas las limitaciones de este, y pensando en poder extenderlo fácilmente en el futuro.

Conceptualmente NTFS ve todo en el sistema como un archivo, incluyendo a los metadatos. El corazón de tal esquema es la **MFT** (Master File Table) que contiene la información de donde están los archivos y sus atributos. Aunque esta respaldada en un %, si se daña, los datos de todo el volumen se perderán.

Utiliza bitmaps para determinar los bloques (clusters) libres en el disco, e indexa los directorios a través de árboles B+.



Tamaño máximo de volumen	16EiB (Exibibyte – 10^{60})
Tamaño máximo de archivo Teórico	16EiB
Tamaño máximo de archivo implementación actual	16TiB (Tebibyte – 10^{40})
Número máximo de archivo	$2^{32} - 1$
Largo nombre de archivo	255 caracteres
Fechas	01/01/1601 – 28/05/60056
Compresión de datos	LZ77 (zip) desde WinNT 3.51
Encriptación de datos	XDES (Win2000), 3DES (WinXP), AES (Win2003)

ESTRUCTURA DEL SISTEMA DE ARCHIVOS.

Cada volumen (partición de disco) de NTFS contiene archivos, directorios, mapas de bits y otras estructuras de datos. Cada volumen se organiza como una secuencia lineal de bloques, en donde el tamaño del bloque está fijo para cada volumen y varía entre 512 bytes y 64 KB, dependiendo del tamaño del volumen. La mayoría de los discos NTFS utilizan bloques de 4 KB como un compromiso entre los bloques grandes (para las transferencias eficientes) y los bloques pequeños (para un nivel bajo de fragmentación interna). Para hacer referencia a los bloques se utiliza su desplazamiento desde el inicio del volumen, mediante el uso de números de 64 bits.

La Figura 12.17 muestra la estructura de un volumen NTFS, que está formado por cuatro regiones. Los primeros sectores de un volumen están ocupados por el sector de arranque de la partición (aunque se llame sector, puede estar formado por un máximo de 16 sectores), que contiene información sobre la estructura del volumen, las estructuras del sistema de ficheros, así como la información de arranque de inicio y el código. Esto es seguido por la tabla maestra de ficheros (Master File Table, MFT), que contiene información sobre todos los ficheros y carpetas (directorios) de este volumen NTFS, así como la información sobre espacio disponible no asignado. En esencia, el MFT es una lista de todos los contenidos de este volumen NTFS, organizada como un conjunto de filas en una estructura de base de datos relacional.

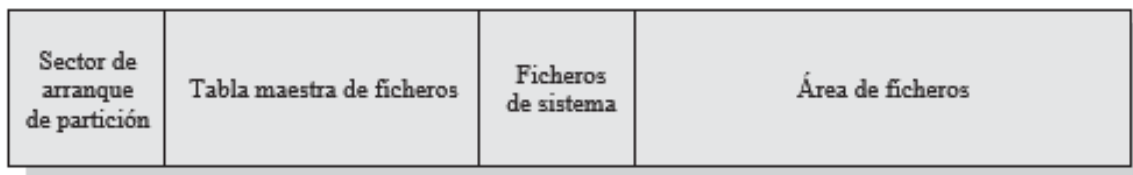


Figura 12.17. Disposición del volumen de NTFS.

Sector de arranque de partición: En los primeros 8kb se contiene la información sobre el volumen (tipo de partición, largo, etc), junto con el bloque de código básico para iniciar al sistema operativo. Contiene también un puntero a la MFT.

MFT – Master File Table: La Tabla Maestra de archivos contiene el dónde y como están almacenados los archivos, junto con todos los atributos asociados a estos.

Archivos de Sistema: Contienen la información sobre los datos y operaciones que se realizan sobre el sistema de archivos: espacio libre, log de transaccionalidad, etc.

Área de archivos: Donde realmente se almacenan los datos del usuario.

Tabla maestra de ficheros MFT

La principal estructura de datos en cada volumen es la MFT (Master File Table, Tabla de archivos maestra), la cual es una secuencia lineal de registros de un tamaño fijo de 1 KB. Cada registro de la MFT describe a un archivo o directorio. Contiene los atributos del archivo, como su nombre y las etiquetas de hora, además de la lista de direcciones de disco en donde se encuentran sus bloques. Si un archivo es muy grande, algunas veces es necesario utilizar dos o más registros de la MFT para contener la lista de todos los bloques, en cuyo caso el primer registro de la MFT (conocido como registro base) apunta a los otros registros de la MFT.

La misma MFT es un archivo, y como tal se puede colocar en cualquier parte dentro el volumen, con lo cual se elimina el problema con los sectores defectuosos en la primera pista. Además, el archivo puede crecer según sea necesario, hasta un tamaño máximo de 2^{48} registros.

Cada registro de la MFT consiste en una secuencia de pares (encabezado de atributo, valor). Cada atributo empieza con un encabezado que le indica qué atributo es, y qué tan largo es el valor. Algunos valores de atributos son de longitud variable, como el nombre del archivo y los datos. Si el valor del atributo es lo bastante corto como para caber en el registro de la MFT, se coloca ahí. A éste se le conoce como archivo inmediato. Si es demasiado largo, se coloca en cualquier otra parte en el disco y se coloca un apuntador a él en el registro de la MFT. Esto hace a NTFS muy eficiente para pequeños campos; es decir, los que pueden caber dentro del mismo registro de la MFT.

Cada uno de los registros describe un archivo normal que tiene atributos y bloques de datos, de igual forma que cualquier otro archivo. Cada uno de estos archivos tiene un nombre que empieza con un signo de dólar, para indicar que es un archivo de metadatos. El primer registro describe el archivo de la MFT. En especial, indica en dónde se encuentran los bloques del archivo de la MFT, de manera que el sistema pueda encontrarlo. Sin duda, Windows necesita una forma de buscar el primer bloque del archivo de la MFT para poder encontrar el resto de la información del sistema de archivos. La forma de encontrar el primer bloque del archivo de la MFT es buscar en el bloque de inicio, en donde se instala su dirección cuando se da formato al volumen con el sistema de archivos.

La MFT contiene la información sobre todos los archivos dentro del volumen. Mantiene la dirección en disco donde está el archivo, sus atributos, información de cómo securizar los datos, etc.

Las primeras 16 posiciones corresponden a entradas a archivos que implementan el sistema de archivos y están reservadas. Desde la 11 a la 15 no se utilizan y están para provisión. Desde el #16 en adelante se pueden tener entradas para archivos de usuario.

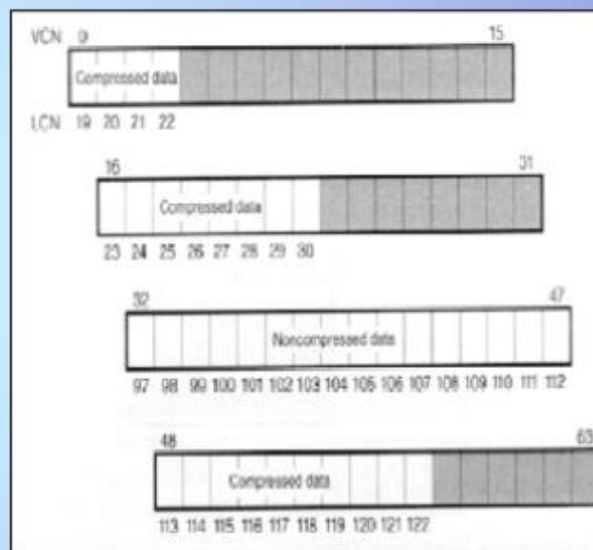
#	Filename	Nombre	Descripción
0	\$MFT	Master File Table	Puntero a sí mismo para consistencia del modelo.
1	\$MFTMirr	Master File Tabla Mirror	Puntero donde se encuentra el respaldo de la MFT. Solo se respaldan los registros más importantes. En NT3.5 se ubica a la mitad de la partición, en versiones posteriores al final.
2	\$LogFile	Log File	Archivo de log de transacciones sobre el volumen.
3	\$Volume	Volume Descriptor	Contiene información crucial sobre la partición en sí: versión NTFS, nombre, etc.
4	\$AttrDef	Attribute Definition Table	Nombres y tipos de los atributos que tendrá cada archivo.
5	\$	Root Directory	Puntero a donde parte el directorio raíz del sistema.
6	\$Bitmap	Cluster Allocation Bitmap	Mapa que muestra los clusters (bloques) disponibles
7	\$Boot	Volume Boot Code	Puntero al segmento que contiene el código de arranque, en el caso de que esta sea una partición activa (bootable)
8	\$BadClus	Bad Cluster File	Lista de todos los bloques "malos" del disco, para no volver a utilizarlos.
9	\$Secure	Security File	Contiene descriptores únicos para cada archivo del volumen
10	\$UpCase	Upper Case Table	Tabla de conversiones mayúsculas/minúsculas en UNICODE.
11	\$Extend	NTFS Extensions	Usado por varias extensiones como cuotas, repase, identif. de objeto, etc.

Compresión de archivos

NTFS implementa un sistema de compresión de datos nativamente (si se estipula que se quiere usar compresión).

NTFS toma el archivo en agrupaciones de 16 bloques y aplica compresión, si el resultado ocupa 15 bloques o menos, se comprime ese grupo (independientemente del resto).

Al acceder a un bloque comprimido, se debe primero descomprimir el grupo donde está para poder acceder a él.



Cifrado de Archivos

NTFS permite el cifrado de los archivos, usando una llave pública (para cifrar) y una privada (para descifrar), al momento de escribir y al momento de leer respectivamente.

Al cifrar combina la llave privada con un numero aleatorio de sistema, de manera de obtener 2 llaves: una para la recuperación de los datos con la llave privada del usuario encriptador (Data Decryption Field – DDF) y otra para que otros usuarios autorizados (incluido el Administrador) puedan descifrar el contenido (Data Recovery Field – DRF).

Administración del espacio en disco

Hay dos estrategias generales para almacenar un archivo: asignar n bytes consecutivos de espacio en disco, o dividir el archivo en bloques, de modo que, si el archivo crece, probablemente tendrá que pasarse a otro lugar del disco. Por esta razón, casi todos los sistemas de archivos dividen los archivos en bloques de tamaño fijo que no necesitan estar adyacentes.

Tamaño de bloque

Una vez que se ha decidido almacenar archivos en bloques de tamaño fijo, se debe decidir qué tamaño deben tener los bloques. Dada la forma como están organizados los discos, el sector, la pista y el cilindro son candidatos para utilizarse como unidad de asignación. Tener una unidad de asignación grande, como un cilindro, implica que cada archivo, incluso un archivo de un byte, ocupará todo un cilindro, desperdiciando espacio. Por otro lado, el empleo de una unidad de asignación pequeña implica que cada archivo consistirá en muchos bloques. La lectura de cada bloque normalmente requiere una búsqueda y un retardo rotacional, así que la lectura de un archivo que consta de muchos bloques pequeños será lenta. El tiempo en milisegundos requerido para leer un bloque de k bytes es la suma de los tiempos de búsqueda, retardo rotacional y transferencia.

En resumen, un tamaño de bloque grande tiene mejor tasa de transferencia, pero desperdicia espacio; un tamaño de bloque pequeño aprovecha el espacio del almacenamiento, pero disminuye el desempeño.

Control de bloques libres

Una vez que se ha escogido el tamaño de bloque, se debe decidir cómo administrar a los bloques libres. Se usan dos técnicas para este control:

- **Lista enlazada de bloques de disco:** en la que cada bloque guarda tantos números de bloques de disco como quepan en él. Con bloques de 1 KB y números de bloque de 32 bits, cada bloque de la lista libre contendrá los números de 255 bloques libres (se necesita una ranura para el puntero al siguiente bloque).

Sólo es preciso mantener un bloque de punteros en la memoria principal. Cuando se crea un archivo tomas los bloques que necesita del bloque de punteros. Cuando el bloque se agota, se lee del disco un nuevo bloque de punteros. Cuando se borra un archivo, sus bloques se liberan y se añaden al bloque de punteros en memoria. Si este bloque se llena, se escribe en el disco. Esto puede generar una cantidad innecesaria de operaciones de E/S de disco (sobre todo los archivos temporales). Para evitar esto, se divide el bloque de punteros lleno.

- **Mapa de bits:** un disco con n bloques requiere un mapa de bits con n bits. Los bloques libres se representan con unos en el mapa y los bloques asignados con ceros (o viceversa).

El mapa de bits requiere menos espacio, ya que utiliza 1 bit por bloque y no 32 bits como la lista enlazada.

Sólo si el disco está casi lleno el método de la lista enlazada requerirá menos bloques que el mapa de bits. Si hay suficiente memoria principal para contener el mapa de bits, este método es preferible. En cambio, si sólo se puede dedicar un bloque de memoria para seguir la pista a los bloques libres, y el disco está casi lleno, la lista enlazada puede ser mejor.

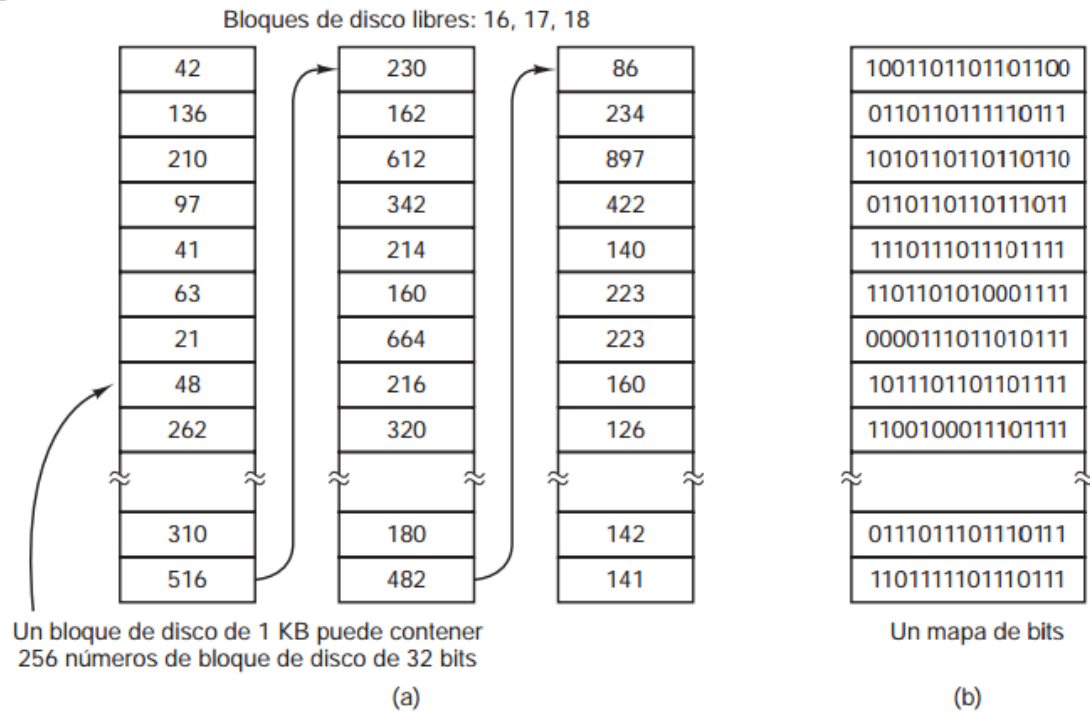


Figura 4-22. (a) Almacenamiento de la lista de bloques libres en una lista enlazada.
(b) Un mapa de bits.

Administración de cuotas de disco

Los SO multiusuario tienen un mecanismo para evitar que las personas acaparen demasiado espacio de disco. El administrador del sistema asigna a cada usuario una porción máxima (límites) de archivos y bloques y el SO cuida que los usuarios no excedan su cuota.

Cuando se abre un archivo, sus atributos y direcciones de disco se colocan en una “tabla de archivos abiertos” en la memoria principal. Cada archivo tiene una entrada que indica quien es el dueño del archivo, por lo que cualquier aumento en el tamaño del archivo se carga automáticamente a la cuota del dueño. Una segunda tabla contiene los registros de cuota de los usuarios que tiene algún archivo abierto en ese momento, para poder localizar los distintos límites. Cada vez que se añade un bloque a un archivo, se incrementa el total de bloques cargados al usuario y se verifica este valor contra los límites estricto y flexible. Se puede exceder el límite flexible, pero no el estricto. Cuando un usuario desea iniciar su sesión, el SO verifica el archivo de cuotas para ver si este se ha excedido su límite flexible (ya sea en número de archivos o bloques). Si se ha excedido, se le advierte (mediante un mensaje) y el contador de advertencias se reduce en uno. Si este llega a cero, no se le permite iniciar sesión nuevamente.

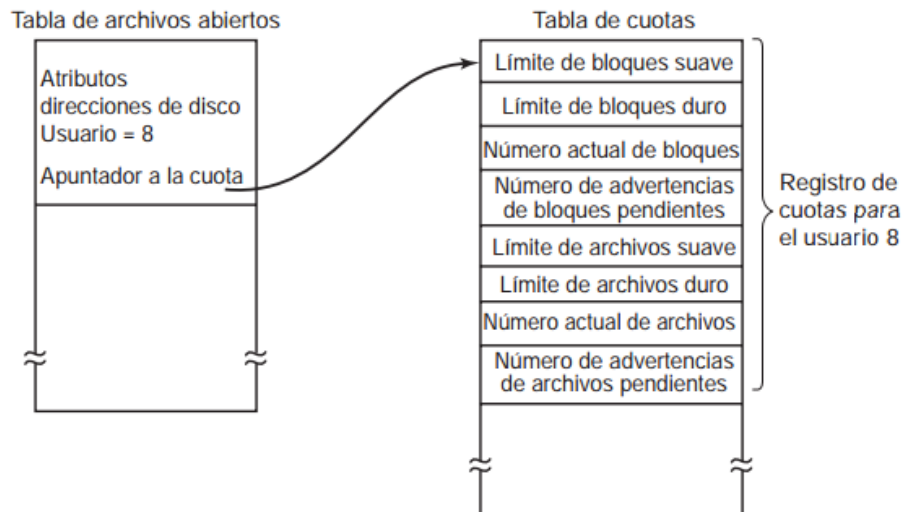


Figura 4-24. El registro de cuotas se lleva en una tabla de cuotas, usuario por usuario.

Unidad N°3 – Administración de procesos

Proceso: es un programa ejecutándose, el programa está en la memoria principal al ejecutarlo, sino está en el disco. Es una unidad de actividad que se caracteriza por la ejecución de una secuencia de instrucciones, un estado actual, y un conjunto de recursos del sistema asociados.

Programa != Proceso

El programa es pasivo. – Código + datos

El proceso es un programa en ejecución. – Pila, registros, contador de programa.

Ejemplo: Ejecutemos dos instancias de Internet Explorer. Mismo programa. Procesos serapados.

En cualquier instante puntual del tiempo, *mientras el proceso está en ejecución*, este proceso se puede caracterizar por una serie de elementos, incluyendo los siguientes:

- **Identificador.** Un identificador único asociado a este proceso, para distinguirlo del resto de procesos.
- **Estado.** Si el proceso está actualmente corriendo, está en el estado *en ejecución*.
- **Prioridad:** Nivel de prioridad relativo al resto de procesos.
- **Contador de programa.** La dirección de la siguiente instrucción del programa que se ejecutará.
- **Punteros a memoria.** Incluye los punteros al código de programa y los datos asociados a dicho proceso, además de cualquier bloque de memoria compartido con otros procesos.
- **Datos de contexto.** Estos son datos que están presentes en los registros del procesador cuando el proceso está corriendo.
- **Información de estado de E/S.** Incluye las peticiones de E/S pendientes, dispositivos de E/S (por ejemplo, una unidad de cinta) asignados a dicho proceso, una lista de los ficheros en uso por el mismo, etc.
- **Información de auditoría.** Puede incluir la cantidad de tiempo de procesador y de tiempo de reloj utilizados, así como los límites de tiempo, registros contables, etc.

La información de la lista anterior se almacena en una estructura de datos, que se suele llamar bloque de control de proceso (*process control block*) (Figura 3.1), que el sistema operativo crea y gestiona.

Identificador
Estado
Prioridad
Contador de programa
Punteros de memoria
Datos de contexto
Información de estado de E/S
Información de auditoría

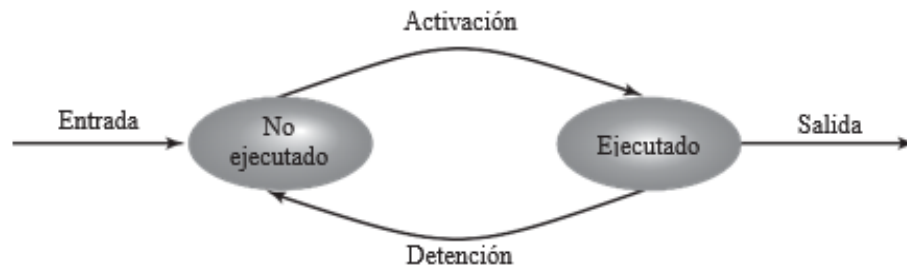
Figura 3.1. Bloque de control de programa (BCP) simplificado.

Estados

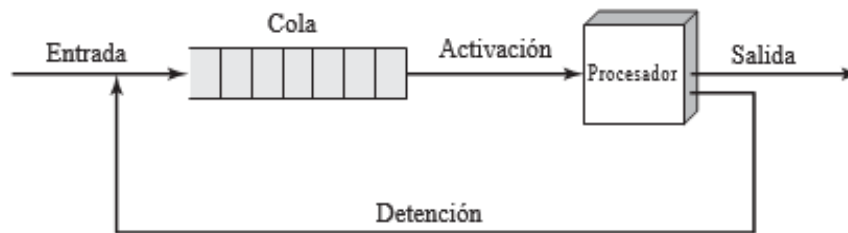
La principal responsabilidad del SO es controlar la ejecución de los procesos, para lo cual hay que describir el comportamiento que se querrá que tengan los procesos.

Modelo de dos estados

Es el modelo más sencillo, teniendo en cuenta que el proceso puede estar ejecutándose en el procesador o no; un proceso puede estar en dos estados: en **Ejecución** o **No Ejecución**. Aquellos procesos que no están en ejecución tienen que guardarse en algún tipo de cola, para que esperen su turno de ejecución.



(a) Diagrama de transiciones de estados



(b) Modelos de colas

Figura 3.5. Modelo de proceso de dos estados. |

Creación de procesos y terminación de procesos

Cuando se añade un proceso a los que ya está administrando el SO, hay que construir las estructuras de datos que se utilizan para esta administración y asignar el espacio de direcciones en la memoria principal para el proceso. Estas acciones constituyen la creación de un proceso.

Cuatro sucesos comunes conducen a la creación de un proceso:

Nuevo proceso de lotes	El sistema operativo dispone de un flujo de control de lotes de trabajos, habitualmente una cinta un disco. Cuando el sistema operativo está listo para procesar un nuevo trabajo, leerá la siguiente secuencia de mandatos de control de trabajos.
Sesión interactiva	Un usuario desde un terminal entra en el sistema.
Creado por el sistema operativo para proporcionar un servicio	El sistema operativo puede crear un proceso para realizar una función en representación de un programa de usuario, sin que el usuario tenga que esperar (por ejemplo, un proceso para controlar la impresión).
Creado por un proceso existente	Por motivos de modularidad o para explotar el paralelismo, un programa de usuario puede ordenar la creación de un número de procesos.

Las causas típicas para la terminación de los procesos son:

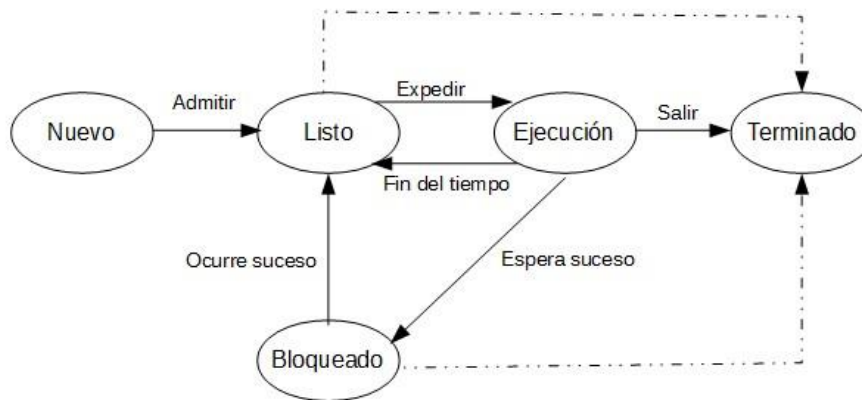
1. En un trabajo por lotes se incluye la instrucción de detención o se hace una llamada al SO.
2. En una aplicación interactiva, es la acción del usuario la que indica cuando termina el proceso.
3. Un proceso puede ser eliminado por un proceso padre o cuando éste termine.
4. También se puede terminar por una serie de fallos o errores.

Modelo de cinco estados

El modelo de dos estados no es adecuado, ya que algunos procesos en el estado de No Ejecución están listos para ejecutarse, mientras que otros están bloqueados esperando a que termina una operación de E/S.

La forma más común de solventar esto es dividir el estado de No Ejecución en dos estados: **Listo** y **Bloqueado**. Luego se incorporan dos estados más por una cuestión de utilidad, quedando cinco estados:

- **Ejecución:** proceso actualmente en ejecución.
- **Listo:** proceso preparado listo para ejecutarse.
- **Bloqueado:** procesos que no se va a ejecutar hasta que se produzca cierto suceso (como una operación de E/S).
- **Nuevo:** proceso que se acaba de crear, pero que no ha sido admitido por el SO en el grupo de procesos ejecutables. Normalmente permanece almacenado en disco.
- **Terminado:** el proceso ha sido excluido del grupo de ejecutables, porque se detuvo o porque fue abandonado por alguna razón. El SO conserva temporalmente las tablas y otra información asociado al proceso terminado para que le dé tiempo a otros programas para extraer información necesaria de él, luego se borran del sistema.



Ahora hay dos colas, una de Listos y una de Bloqueados. A medida que se admiten procesos en el sistema, se sitúan en la cola de Listos. Cuando llega la hora de que el SO escoja otro proceso para ejecutar, selecciona uno de la cola de Listos (cola FIFO). Cuando se produce un suceso, todos los procesos de la cola Bloqueados que están esperando a dicho suceso se pasan a la cola de Listos. El SO debe recorrer toda la cola de Bloqueados, buscando aquellos procesos que esperan el suceso, pudiendo haber cientos o incluso miles de procesos en dicha cola. Sería más eficiente tener una cola para cada suceso.

Si la expedición de procesos está dictada por un esquema de prioridades, entonces es conveniente tener un cierto número de colas de Listos, una para cada nivel de prioridad. El SO podrá determinar fácilmente cuál es el proceso de prioridad más alta que lleva más tiempo esperando.

Transiciones

- **Nulo** → **Nuevo**: se crea un nuevo proceso para ejecutar un programa.
- **Nuevo** → **Listo**: el SO está preparado para aceptar un proceso más.
- **Listo** → **Ejecución**: momento de seleccionar un nuevo proceso para ejecutar, el SO elige uno de los procesos del estado Listo.
- **Ejecución** → **Terminado**: el proceso en ejecución es finalizado por el SO.
- **Ejecución** → **Listo**: la razón más común es que el proceso en ejecución ha alcanzado el tiempo máximo permitido de ejecución ininterrumpida. Hay otras causas, como los niveles de prioridad o que un proceso ceda voluntariamente el control del procesador.
- **Ejecución** → **Bloqueado**: un proceso solicita por algo por lo que debe esperar.

- **Bloqueado** → **Listo**: se produce el suceso que el proceso estaba esperando.
- **Listo** → **Terminado**: por ejemplo, un padre termina con un proceso hijo, o el mismo padre termina, terminando con él todos sus hijos.
- **Bloqueado** → **Terminado**: mismo caso de Listo → Terminado.

Procesos suspendidos

Hay una buena justificación para agregar más estados al modelo. Las actividades de E/S son mucho más lentas que las de cálculo y, por lo tanto, el procesador en un sistema de monoprogramación está la mayor parte del tiempo desocupado. El procesador es tan rápido comparado con la E/S que suele ser habitual que todos los procesos de memoria estén esperando por E/S. Incluso con multiprogramación, el procesador podría estar desocupado la mayor parte del tiempo.

Una solución es el **intercambio**, o sea, mover una parte del proceso o todo el proceso de la memoria principal al disco. Cuando ninguno de los procesos en la memoria principal está en estado Listo, el SO pasa al disco uno de los procesos en Bloqueado y lo lleva a una cola de Suspendidos. El SO trae entonces otro proceso de la cola de Suspendidos o acepta la solicitud de creación de un nuevo proceso. La ejecución continúa con el proceso recién llegado.

No obstante, el intercambio es una operación de E/S, pero como la E/S con el disco es la E/S más rápida de un sistema (comparado con una impresora o una cinta) suele mejorar el rendimiento.

Podemos analizar el concepto de procesos suspendidos, definiendo un proceso suspendido como el que cumple las siguientes características:

- 1.El proceso no está inmediatamente disponible para su ejecución.
- 2.El proceso puede estar o no a la espera de un evento, si es así, la condición de bloqueo es independiente de la condición estar suspendido, y si sucede el evento que lo bloquea, eso no habilita al proceso para su ejecución inmediata.
- 3.El proceso fue puesto en estado suspendido por un agente: bien el proceso mismo, el proceso padre o el sistema operativo, con el propósito de prevenir su ejecución.
- 4.El proceso no puede ser recuperado de este estado hasta que el agente explícitamente así lo indique.

Modelo de siete estados

Al emplear el intercambio, se debe añadir un nuevo estado al modelo del comportamiento de los procesos, el estado **Suspendido**. A su vez, un proceso en estado Suspendido puede estar Bloqueado o Listo, por eso se necesitan cuatro estados:

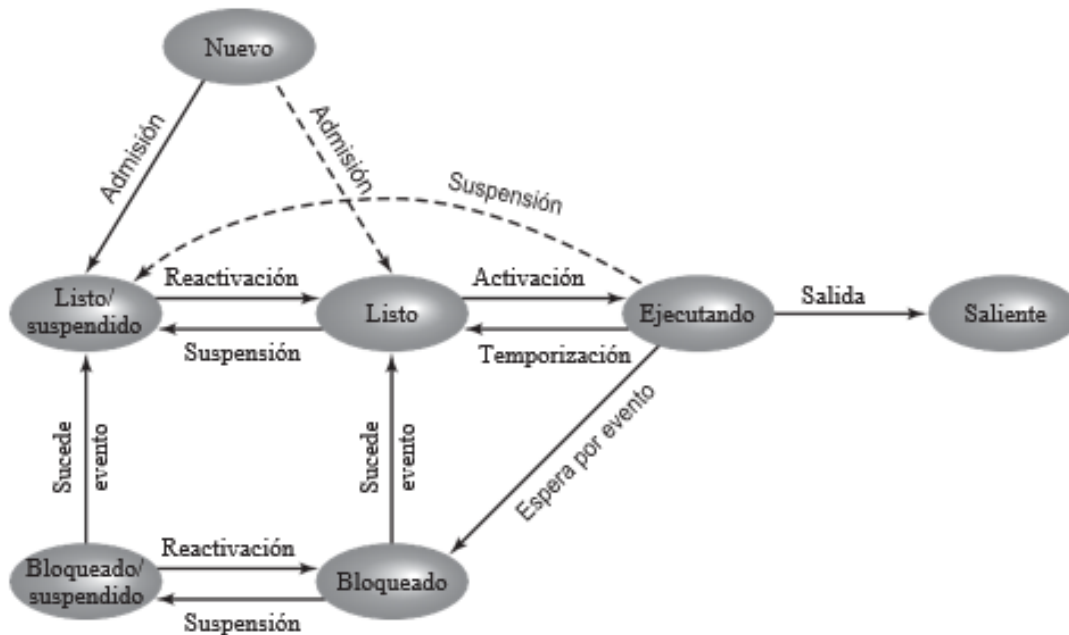
- **Listo**: proceso en memoria principal y listo para la ejecución.
- **Bloqueado**: proceso en memoria principal, esperando un suceso.
- **Bloqueado y suspendido**: proceso en memoria secundaria esperando un suceso.
- **Listo y suspendido**: proceso en memoria secundaria disponible para la ejecución en cuanto se cargue en la memoria principal.

Transiciones

- **Bloqueado** → **Bloqueado y suspendido**: si no hay procesos Listos, entonces al menos un proceso Bloqueado se expulsa para dar cabida a otro proceso que no esté Bloqueado.
- **Bloqueado y suspendido** → **Listo y suspendido**: se produce un suceso que estaba esperando en memoria secundaria.
- **Listo y suspendido** → **Listo**: cuando no hay procesos Listos en memoria principal, el SO tendrá que traer uno para continuar la ejecución.
- **Listo** → **Listo y suspendido**: puede ser necesario suspender un proceso Listo si ésta es la única forma de liberar espacio en memoria principal. El SO puede suspender un proceso Listo de más baja prioridad en lugar de uno Bloqueado que sea de prioridad más alta, si cree que el proceso Bloqueado pronto estará Listo.
- **Nuevo** → **Listo y suspendido**, **Nuevo** → **Listo**: cuando se crea un nuevo proceso, se lo puede añadir a la cola de Listos o a la de Listos y suspendidos. En el segundo caso puede deberse a poco espacio en memoria principal.
- **Bloqueado y suspendido** → **Bloqueado**: un proceso termina y se libera memoria; hay un proceso en Bloqueados

y suspendidos que tiene una prioridad mayor a la de cualquier proceso de Listos y suspendidos, así que el SO supone que ocurrirá el suceso que espera pronto.

- **Ejecución → Listo y suspendido:** hay un proceso en Bloqueados y suspendidos de prioridad alta que se acaba de desbloquear, para ello expulsa al proceso en ejecución y lo suspende.
- **Varios → Terminado:** un proceso en cualquier estado puede pasar al estado Terminado.



Descripción de procesos

Estructuras de control del sistema operativo

Si el SO va a administrar los procesos y los recursos, entonces tiene que disponer de información sobre el estado actual de cada proceso y de cada recurso; para ello, el SO construye y mantiene tablas de información sobre cada entidad que está administrando. Los SO organizan la información principalmente en cuatro categorías:

- **Tablas de memoria:** se utiliza para seguir la pista de la memoria principal (real) y secundaria (virtual). Parte de la memoria principal está reservada para uso del SO, el resto es para los procesos. Incluyen información como asignación de memoria principal y secundaria a los procesos, información de protección de regiones compartidas (por los procesos) de memoria.
- **Tablas de dispositivos de E/S:** utilizadas por los SO para administrar los dispositivos y canales de E/S, los cuales pueden estar disponibles o asignados a un proceso en particular.
- **Tablas de archivos:** ofrecen información sobre la existencia de los archivos, su posición en la memoria secundaria, su estado actual y otros atributos.
- **Tablas de procesos:** sirve para administrar los procesos. Se genera un número para cada proceso, para poder identificarlo.

Estas tablas están enlazadas o relacionadas, disponen de referencias cruzadas. La memoria, la E/S y los archivos son administrados en nombre de los procesos, por lo que debe haber alguna referencia directa o indirecta a estos recursos en las tablas de procesos.

Estructuras de control de procesos

Lo que un SO debe saber para administrar y controlar los procesos es: dónde está el proceso y conocer los atributos de éste.

Bloque de control de proceso e Imagen de proceso

Un proceso constará, al menos, de la memoria suficiente para albergar los programas y los datos del proceso. Además de esto, en la ejecución de un programa entra en juego normalmente una pila, que se utiliza para llevar la cuenta de las llamadas a procedimientos y de los parámetros que se pasan entre los procedimientos. Por último, asociado a cada proceso hay una

serie de atributos utilizados por el SO para su control. Estos atributos se conocen como **bloque de control del proceso (PCB)**. Esta colección de programas, datos, pila y atributos se conoce como **imagen del proceso**.

En el caso más simple, la imagen del proceso se guarda como un bloque contiguo, o continuo de memoria. Este bloque se mantiene en memoria secundaria, habitualmente disco. Para que el SO pueda administrar el proceso, al menos una pequeña parte de su imagen, que contiene la información a usar por el SO, debe mantenerse en memoria principal. Para ejecutar el proceso, la imagen completa debe cargarse a la memoria principal, o al menos a la memoria virtual. Por lo tanto, las tablas mantenidas por el SO, deben mostrar la ubicación de cada segmento o página (dependiendo el esquema que se use) de cada imagen de proceso.

Los SO modernos utilizan un esquema de gestión de memoria en donde la imagen de un proceso consiste en un conjunto de bloques que no tienen por qué estar almacenados consecutivamente. Esto permite al SO tener que traer solo una parte de un proceso en particular. De este modo, en un momento dado, una parte de la imagen de un proceso puede estar en la memoria principal y el resto en la memoria secundaria. Las tablas de procesos mantenidas por el SO, deben mostrar la ubicación de cada segmento o página de cada imagen de proceso.

Atributos de los procesos

La información de los BCP se puede agrupar en tres categorías:

- **Identificación del proceso:** cada proceso tiene un identificador (PID).
- **Información de estado del procesador:** formada por el contenido de los registros del procesador.
- **Información de control del proceso:** información adicional para que el SO controle y coordine los procesos activos.

La importancia del PCB

El PCB es la estructura de datos más importante de un SO. Cada PCB contiene toda la información de un proceso necesaria para el SO. Puede decirse que el conjunto de los PCB definen el estado del SO.

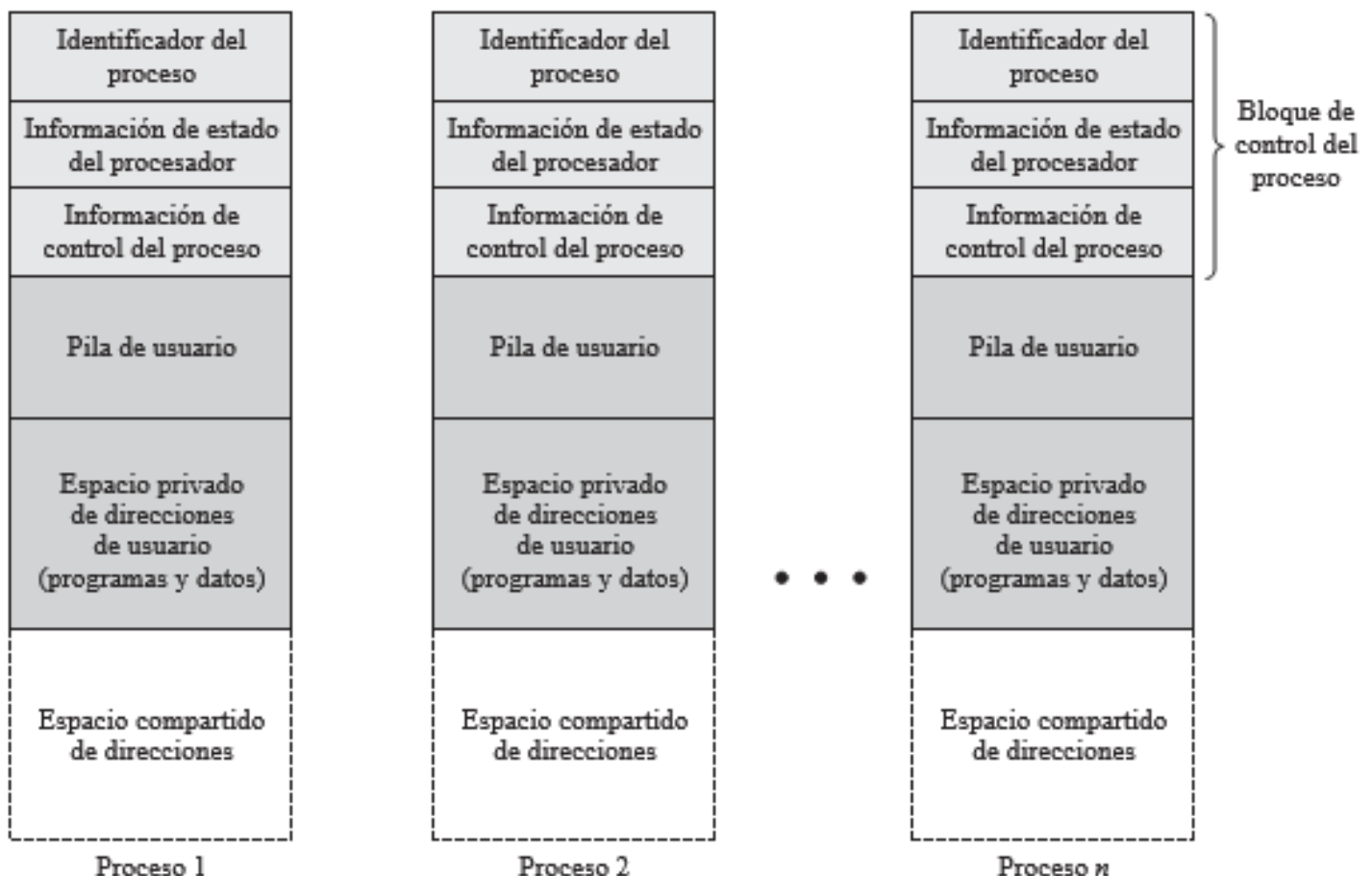


Figura 3.13. Procesos de usuario en memoria virtual.

Control de procesos

Modos de ejecución

- **Modo de usuario:** Ofrece un conjunto limitado de instrucciones que no permiten acceder a determinados elementos del sistema. Para cambiar a modo usuario es “sencillo”, consiste en perder privilegios.
- **Modo núcleo:** Aquí el software tiene control completo sobre el procesador y de todas sus instrucciones, registros y memoria. El cambio a modo núcleo se realiza cuando un proceso ejecuta una llamada al sistema o interrupción.

La razón de usar dos modos, es porque es necesario proteger al SO y a las tablas importantes del mismo. Existe un bit en la palabra del estado del programa (PSW) que indica al procesador el modo de ejecución. Para cambiar el modo de ejecución se cambia el bit. El SO ejecuta en modo núcleo y el resto de los procesos lo hacen en modo usuario.

Funciones del núcleo de un SO

- **Gestión de procesos:** creación y terminación, planificación y activación de procesos, intercambio de procesos, gestión de PCB, sincronización de procesos, y soporte para la comunicación entre ellos.
- **Gestión de memoria:** reserva de espacio de direcciones para los procesos, intercambio, gestión de páginas y segmentos.
- **Gestión de E/S:** gestión de buffers, asignación de canales de E/S y dispositivos para los procesos.
- **Funciones de soporte:** tratamiento de interrupciones, auditoria, supervisión.

El motivo por el cual se usan los otros modos es claro. Se necesita proteger al sistema operativo y a las tablas clave del sistema, por ejemplo, los bloques de control de proceso, de la interferencia con programas de usuario. En modo núcleo, el software tiene control completo del procesador y de sus instrucciones, registros, y memoria. Este nivel de control no es necesario y por seguridad no es recomendable para los programas de usuario.

Creación de procesos

La creación de un proceso incluye los siguientes pasos:

1. **Asignar un identificador de proceso único al proceso:** se añade una nueva entrada a la tabla principal de procesos.
2. **Reservar espacio para el proceso:** incluye todos los elementos de la imagen del proceso. Se debe asignar espacio para el PCB.
3. **Iniciar el PCB:** la información del estado del procesador se inicia generalmente con todas las entradas en cero excepto por el contador de programa y los punteros de pila del programa. La información de control del procesador se inicia a partir de los valores estándares por omisión y los atributos que se han solicitado para el proceso.
4. **Establecer los enlaces apropiados:** por ejemplo, poner el nuevo proceso en la cola de Listos o Listos y suspendidos.
5. **Crear o ampliar otras estructuras de datos:** por ejemplo, el SO puede mantener un registro de auditoria por cada proceso que se puede usar más adelante a efectos de facturación y/o de análisis de rendimiento del sistema.

Cambio de proceso

Un cambio de proceso puede producirse en cualquier momento en que el SO haya tomado el control a partir del proceso que está actualmente ejecutándose. Los posibles sucesos que pueden dar el control al SO son:

- **Interrupción:** originada por un suceso externo e independiente del proceso en ejecución. El control se transfiere a un gestor de interrupciones quien se encarga del tipo de interrupción ocurrida:
 - Interrupción de reloj: el SO determina si el proceso ha consumido el tiempo máximo permitido. Si esto pasa, el proceso que se está ejecutando pasa a Listo y se debe expedir otro proceso.
 - Interrupción de E/S: el SO determina que acción de E/S se ha producido. Si es un suceso que muchos procesos estaban esperando, se los pasa a Listo, luego el SO debe decidir si reanudar la ejecución o ejecutar uno de los procesos Listos de mayor prioridad.
 - Fallo de memoria: referencias a memoria virtual de una palabra que no está en memoria principal. El proceso que cometió el fallo pasa a estar Bloqueado hasta que se cargue el bloque en memoria, donde cambia a Listo.
- **Trap:** condición de error o excepción generada dentro del proceso en ejecución. Si es un error fatal, el proceso pasa a Terminado. Si no es fatal, depende del error y del SO, puede intentar una recuperación o informar al usuario.
- **Llamada al sistema:** el uso de una llamada al sistema hace que el proceso de usuario pase al estado Bloqueado y el control pase al SO.

CAMBIO DE CONTEXTO:

- Salvar el contexto del proceso.
- Analizar el tipo de interrupción producida.
- Asignar al contador de programa la dirección de la rutina de tratamiento de la interrupción.
- Cambiar de estado *usuario* a estado *sistema*.
 - CHM
 - Cambiar a PSW de la rutina a ejecutar.

CAMBIO DE PROCESO:

- Salvar el contexto del proceso.
- Actualizar el PCB del proceso en ejecución, cambiando el estado.
- Mover este PCB a la cola apropiada según el estado.
- Seleccionar otro proceso para ejecución.
- Actualizar el PCB del proceso seleccionado indicando el estado *ejecución*.
- Actualizar las estructuras de datos de gestión de memoria.
- Restaurar el contexto del procesador.

- Cuando la CPU conmuta a otro proceso, el sistema debe salvar el estado del viejo proceso y cargar el estado para el nuevo proceso vía un cambio de contexto
- El contexto de un proceso está representado en el PCB
- El tiempo que lleva el cambio de contexto es sobrecarga; el sistema no hace trabajo útil mientras está conmutando.
- El tiempo depende del soporte de hardware.

Conmutación de CPU de Proceso a Proceso